

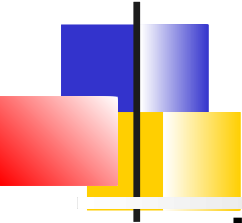


Spring Boot

Accélérez vos développements avec
Spring Boot

David THIBAU – 2025

david.thibau@gmail.com



Agenda

Introduction

- Historique
- IoC et Dependency Injection

Annotations

- Classes de configuration
- @Component et stéréotypes
- Injection de dépendances
- Environnement et profils

Spring Boot

- Principes de l'auto-configuration
- Apports des starters
- Configuration SpringBoot

Spring et la persistance

- Spring Data
- Spring Data JPA

APIs REST

- Spring MVC et les APIs Rest
- Spring Boot et APIs Rest
- Jackson et la désérialisation
- Exceptions, CORS, OpenAP

Interactions entre services

- RestClient
- Messaging

Spring Security

- Principes, modèle stateful,stateless
- Autoconfiguration SpringBoot
- OpenId et oAuth2

Spring et les tests

- Spring Test, @SpringBootTest
- Tests autoconfigurés

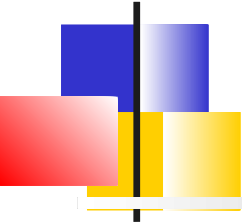
Vers la production

- Actuator
- Packaging et déploiement

Annexes

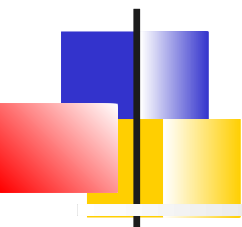
NoSQL

Spring Reactive



Introduction

Historique IoC et Dependency Injection

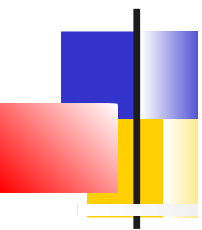


Historique

Spring est un projet OpenSource avec un modèle économique basé sur le service

(Support, Conseil, Formation, Partenariat et certifications)

La société Interface21 fondé par les créateur de Spring Rod Johnson et Juergen Hoeller a été de nombreuses fois racheté par SpringSource, VMWare et maintenant Broadcom

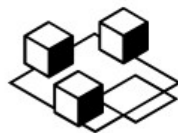


What Spring can do



Generative AI

Integrate AI into your Spring applications without reinventing the wheel.



Microservices

Quickly deliver production-grade features with independently evolvable microservices.



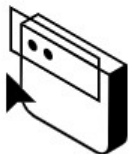
Reactive

Spring's asynchronous, nonblocking architecture means you can get more from your computing resources.



Cloud

Your code, any cloud—we've got you covered. Connect and scale your services, whatever your platform.



Web apps

Frameworks for fast, secure, and responsive web applications connected to any data store.



Serverless

The ultimate flexibility. Scale up on demand and scale to zero when there's no demand.



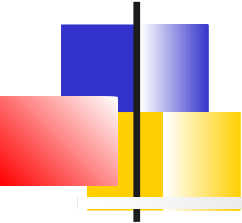
Event Driven

Integrate with your enterprise. React to business events. Act on your streaming data in realtime.



Batch

Automated tasks. Offline processing of data at a time to suit you.



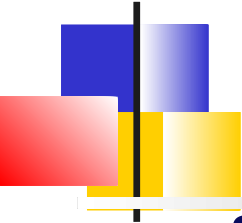
Projets Spring

Spring est donc un ensemble de projets adaptés à toutes les problématiques actuelles.

Tous ces projets ont comme objectifs :

- Permettre d'écrire du code propre, modulaire et testable
- Éviter d'avoir à coder les aspects techniques
- Être portable : Seuls pré-requis : JVM ou Cloud
- Être productif : Fournir les outils de productivité aux développeurs

Tous ces projets s'appuient sur un principe :
l'IoC et l'injection de dépendances



Quelques projets

Spring core : Coeur du framework. Pattern IoC et services de bas niveau

Spring Security : Sécuriser les applications Java et plus particulièrement web

Spring Data : Approche commune pour accéder à des supports persistants (SQL, NOSQL)

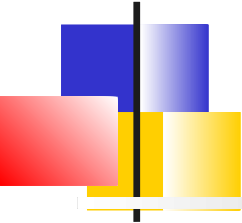
Spring Integration : Intégration d'applications legacy

Spring Batch : Optimisation des travaux par lots (batch)

Spring Cloud : Déploiement des micro-services sur le cloud

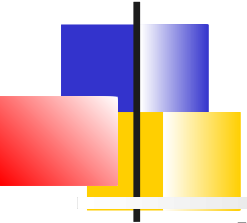
Spring AI : Intégration de l'IA générative dans les applications

...



Introduction

Historique **IoC et Dependency Injection**



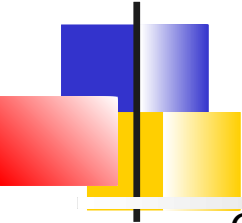
Pattern IoC

Le problème :

Comment faire fonctionner l'architecture web basée sur des contrôleurs avec la base de données quand ceux-ci sont développés par des équipes différentes ?

Réponse de tout développeur POO :

En définissant une interface, bien sûr



Illustration

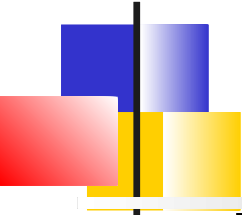
On veut implémenter un contrôleur permettant de lister tous les films d'un metteur en scène donné. Cette classe pourra s'appuyer sur la couche d'accès aux données qui permet de récupérer tous les films d'une base de données :

L'interface définie par l'équipe BD est :

```
public interface MovieDao {  
    List<Movie> findAll();  
}
```

En se basant sur ce contrat, on peut implémenter la méthode :

```
public class MovieController...  
public List<Movie> moviesDirectedBy(String arg) {  
    List<Movie> allMovies = finder.findAll();  
    return allMovies.stream()  
        .filter(m -> !m.getDirector().equals(arg))  
        .collect(Collectors.toList()) ;  
}
```

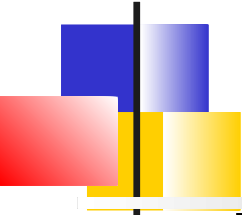


Implémentation ?

Le code est bien découplé, mais pour l'exécuter, il faut une classe concrète qui implémente l'interface.

Par exemple, on peut instancier une implémentation dans le constructeur de la classe contrôleur.

```
public class MovieController...  
private MovieFinder finder;  
public MovieController() {  
    // Argh, au secours on devient dépendant de l'implémentation  
    dao = new ColonDelimitedMovieDao("movies1.txt");  
}
```

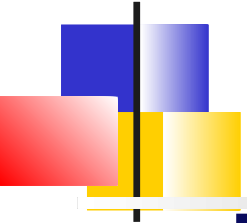


IoC et Framework

Le pattern ***IoC (Inversion Of Control)*** signifie que ce n'est plus le code du développeur qui a le contrôle de l'exécution mais le framework

Le framework est alors responsable d'instancier les objets, d'initialiser les dépendances, d'appeler les méthodes, de libérer les objets, etc..

Le code du développeur est réduit au code métier.

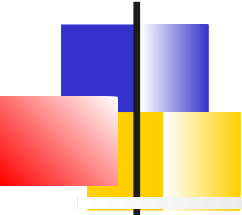


Dependency Injection

L'injection de dépendance est une spécialisation du pattern IoC

Le framework a la responsabilité de construire les instances et d'initialiser leurs attributs .

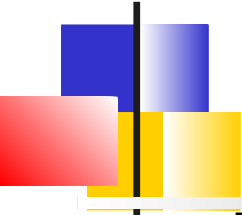
Dans l'illustration précédente, il initialise la variable d'interface avec une implémentation.



Types d'injection de dépendances

On distingue différentes façons d'injecter les dépendances :

- Via le **constructeur** : Les attributs sont initialisés dans le constructeur
- Via une méthode **setter** : Les attributs sont initialisés via une méthode setter
- Via une **méthode arbitraire** : Les attributs sont initialisés via une méthode spécifique
- Directement via les **attributs** : Les attributs sont directement mis à jour par le framework



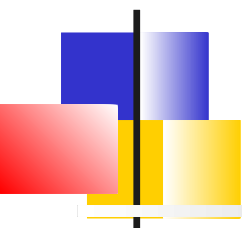
Injection par constructeur

La classe *MovieController* doit déclarer un constructeur permettant d'initialiser l'attribut *MovieDao*.

```
class MovieController...  
public MovieController(MovieDao movieDao) {  
    this.movieDao = movieDao;  
}
```

L'objet dao se fait injecter une **propriété** via le constructeur : le nom du fichier .

```
class ColonMovieDao...  
public ColonMovieDao(String filename) {  
    this.filename = filename;  
}
```

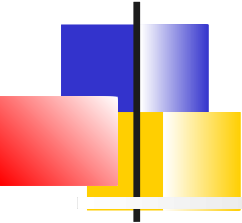


Configuration du framework

La configuration du framework consiste à lui préciser les classes qu'il doit gérer (beans) et comment il injecte les dépendances.

Cela s'effectue :

- Via des fichiers de configuration XML.(Legacy)
- Via des classes Java de configuration et des annotations Java (Depuis Java5)
- En appliquant des configurations par défaut (SpringBoot)



Configuration XML

```
<beans>
```

```
  <bean id="movieController" class="spring.MovieController">
```

```
    <property name="movieDao" ref="colonMovieDao"/>
```

```
  </bean>
```

```
  <bean id="colonMovieDao" class="spring.ColonMovieFinder">
```

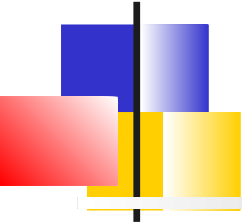
```
    <property name="filename">
```

```
      <value>movies1.txt</value>
```

```
    </property>
```

```
  </bean>
```

```
</beans>
```

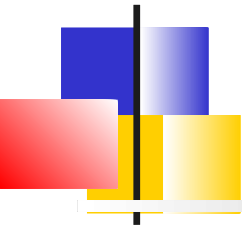


BeanDefinition

Le framework agit comme un container de beans

A l'intérieur du container, les beans sont représentés par la classe ***BeanDefinition*** qui encapsule :

- Le nom complet de la classe du bean
- Son cycle de vie (scope, méthodes de call-back ...).
- Des références vers les autres beans (dépendances)
- ...

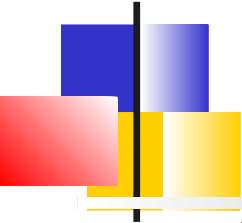


Nommage

Chaque bean possède un (ou plusieurs) identifiant : son nom.

La convention consiste à utiliser la convention de dénomination Java standard pour un attribut d'instance.

Les autres noms sont appelés alias.



ApplicationContext

A la lecture de sa configuration, *Spring* construit un objet ***ApplicationContext*** qui contient les définitions de Bean

Le contexte applicatif expose les méthodes :

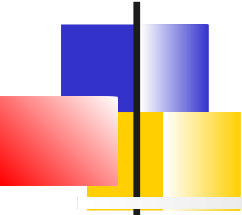
Object getBean(String name) : Retourne une instance de bean par son nom.

<T> T getBean(Class<T> requiredType) : Renvoie l'instance du bean qui correspond de manière unique au type d'objet donné.

Class<?> getType(String name) : Détermine le type du bean portant le nom donné.

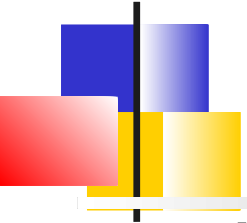
boolean isSingleton(String name) : Ce bean est-il un singleton partagé ?

...



Exemple Test du contrôleur

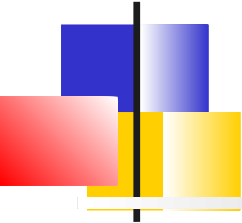
```
public void testWithSpring() throws Exception {  
    // Initialisation du framework via XML  
    ApplicationContext ctx = new FileSystemXmlApplicationContext("spring.xml");  
    MovieController lister = (MovieController)ctx.getBean("movieController");  
  
    // Exécution  
    Movie[] movies = lister.moviesDirectedBy("Sergio Leone");  
    assertEquals("Once Upon a Time in the West",movies[0].getTitle());  
}
```



Aspects (AOP)

Lors de la configuration, il est également possible d'ajouter des services techniques/transverses aux méthodes des objets appelés par le framework :

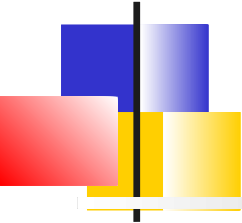
- Sécurité : @RolesAllowed
- Transaction : @Transactional
- ..



Conséquences

Avec un framework IoC comme Spring, un développeur peut :

- Écrire une méthode s'exécutant dans une transaction base de données sans utiliser l'API de transaction
- Rendre une méthode accessible à distance sans utiliser une API remote
- Définir une méthode protégée par des ACLs sans utiliser de code de l'API de sécurité
- Définir une méthode gestionnaire de message sans utiliser l'API du broker



Configuration via les annotations

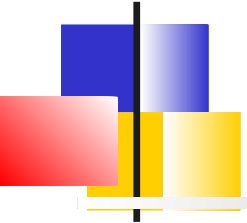
Classes de configuration

@Component et stéréotypes

Injection de dépendances

Propriétés de configuration

Environnement et profils

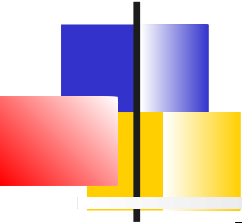


Comparaison avec XML

A la place du XML, il est possible d'utiliser des annotations et des classes de configuration Java pour définir les beans à instancier.

Chaque approche a ses pour et ses contres.
Elles peuvent également être combinées.

Dans les faits, la configuration via Java est beaucoup plus pratique et s'est donc imposée.



Classes de configuration

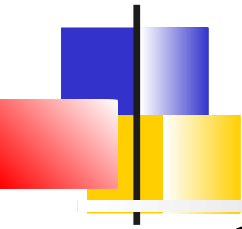
Depuis la 3.0, la configuration peut s'effectuer via des classes Java annotées par **@Configuration**

Ces classes contiennent de méthodes annotées par **@Bean** qui instancient les implémentations d'interface

Ces techniques est utilisée pour instancier les beans d'intégration aux autres systèmes (DataSource, Client Elasticsearch,)

@Configuration

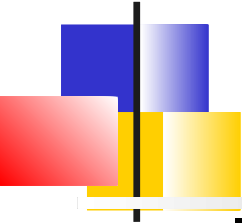
```
public class AppConfig {  
    @Bean  
    public MyService myService() {  
        return new MyServiceImpl();  
    }  
}
```



Lecture de la configuration

2 moyens afin que le framework traite les annotations :

- Lui indiquer l'emplacement des classes de configuration
- Lui indiquer un package à scanner pour découvrir les annotations



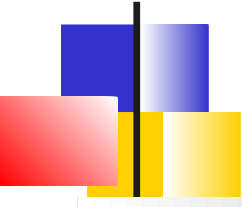
Configuration du framework via les annotations

Indication d'une classe de @Configuration

```
public static void main(String[] args) {  
    ApplicationContext ctx =  
        new AnnotationConfigApplicationContext(AppConfig.class);  
    MyService myService = ctx.getBean(MyService.class);  
    myService.doStuff();  
}
```

Scan de package :

```
public static void main(String[] args) {  
    AnnotationConfigApplicationContext ctx =  
        new AnnotationConfigApplicationContext();  
    ctx.scan("org.formation");  
    ctx.refresh();  
    MyService myService = ctx.getBean(MyService.class);  
}
```



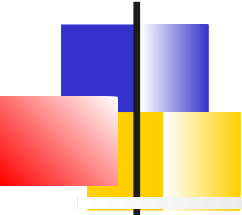
Composition de configuration

L'annotation `@Import` permet d'inclure une configuration dans une autre classe
`@Configuration`

```
@Configuration
public class ConfigA {
    public @Bean A a() { return new A(); }
}
```

```
@Configuration
@Import(ConfigA.class)
public class ConfigB {
    public @Bean B b() { return new B(); }
}
```

```
ApplicationContext ctx = new
    AnnotationConfigApplicationContext(ConfigB.class);
```

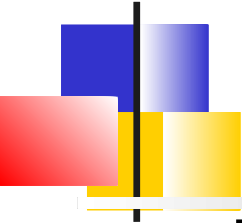


Attributs de *@Bean*

@Bean a 3 attributs :

- ***name*** : les alias du bean
- ***init-method*** : Méthode appelée après l'initialisation du bean par Spring
- ***destroy-method*** : Méthode appelée avant la destruction du bean par Spring

```
@Configuration
public class AppConfig {
    @Bean(name={"foo", "super-foo"}, initMethod = "init")
    public Foo foo() {
        return new Foo();
    }
    @Bean(destroyMethod = "cleanup")
    public Bar bar() {
        return new Bar();
    }
}
```

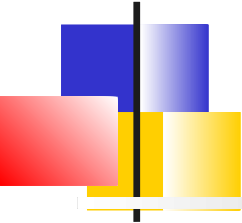


Annotations *@Enable*

Les classes @Configuration sont généralement utilisées pour configurer des ressources techniques (DataSource, RestClient, ...)

Pour faciliter la configuration de ces ressources, Spring fournit des annotations **@Enable** qui configurent les valeurs par défaut suffisantes dans un environnement de dev.

La configuration par défaut peut ensuite être surchargées.



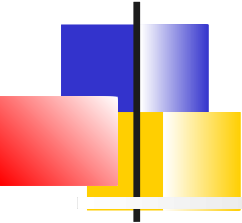
Exemples *@Enable*

@EnableWebMvc : Instancie tous les beans nécessaires à Spring MVC

@EnableCaching : Configuration par défaut d'un cache. Permet d'utiliser les annotations *@Cacheable*, ...

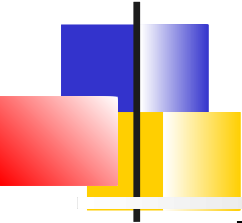
@EnableScheduling : Configuration par défaut d'un scheduler. Permet d'utiliser les annotations *@Scheduled*

@EnableJpaRepositories : Permet de scanner les classe Repository



Configuration via les annotations

Classes de configuration
@Component et stéréotypes
Injection de dépendances
Propriétés de configuration
Environnement et profils

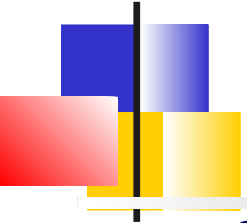


Introduction

Des annotations sont également utilisées pour marquer une classe comme bean Spring.

- L'instanciation du bean est alors fait par le framework.
- Il s'agit en général de beans métier.

L'annotation générique est **@Component**, placée sur la classe



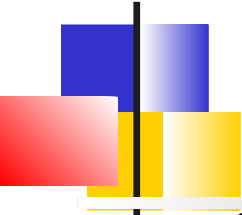
Exemple

@Component

```
public class SimpleMovieLister {  
  
    private final MovieDao movieDao;  
  
    public SimpleMovieLister(MovieDao movieDao) {  
        this.movieDao = movieDao;  
    }  
}
```

@Component

```
public class JpaMovieDao implements MovieDao {  
    ...  
}
```



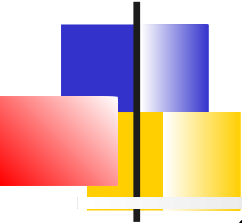
@ComponentScan

Spring peut détecter automatiquement les classes annotées

Il suffit d'utiliser l'annotation **@ComponentScan** en indiquant un package.

Cela s'effectue habituellement sur une classe *@Configuration*

```
@Configuration
@ComponentScan(basePackages = "org.example")
public class AppConfig {
    ...
}
```



Stereotypes

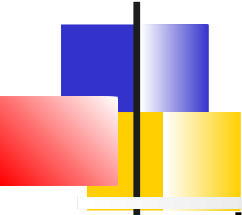
@Component est un stéréotype générique pour tous les composants gérés par Spring.

Spring introduit d'autres stéréotypes :

@Repository, ***@Service***, ***@Controller*** et ***@RestController*** sont des spécialisations de *@Component* pour des cas d'usage plus spécifique (persistance, service, et couche de présentation)

Les stéréotypes servent à classer les beans et éventuellement à leur ajouter des comportements génériques.

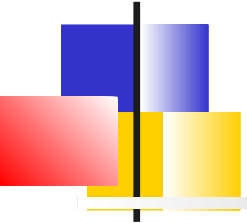
Par ex : La sérialisation JSON pour les @RestController



Cycles de vie

Les beans qu'ils soient instanciés via les classes de Configuration ou les annotations stéréotypées peuvent avoir 3 cycles de vie (ou scope) :

- **Singleton**: Il existe une seule instance de l'objet (qui est donc partagé). Idéal pour des beans « stateless ».
=> C'est l'écrasante majorité des cas.
- **Prototype** : A chaque fois que le bean est utilisé via son nom, une nouvelle instance est créée.
=> Quasiment Jamais
- **Custom object "scopes"** : Leur cycle de vie est généralement synchronisé avec d'autres objets, comme une requête HTTP, une session http, une transaction BD
=> Certains beans fournis par Spring. Ex : EntityManager



@Scope

L'annotation **@Scope** permet de préciser un des scopes prédéfinis de Spring ou un scope personnalisé

```
@Scope("prototype")
```

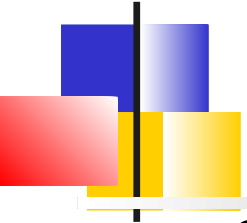
```
@Repository
```

```
public class MovieFinderImpl implements
```

```
MovieFinder {
```

```
// ...
```

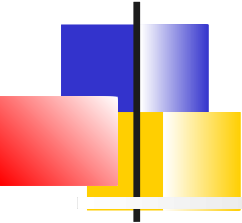
```
}
```



Méthodes de call-back

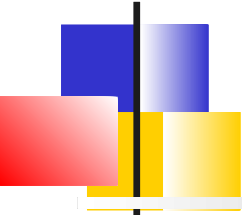
Spring supporte les annotations de call-back
@PostConstruct et **@PreDestroy**

```
@Component
public class CachingMovieLister {
    @PostConstruct
    public void populateMovieCache() {
        // Initialisation après construction...
    }
    @PreDestroy
    public void clearMovieCache() {
        // Nettoyage avant destruction...
    }
}
```

Configuration via les annotations

Classes de configuration
@Component et stéréotypes
Injection de dépendances
Propriétés de configuration
Environnement et profils

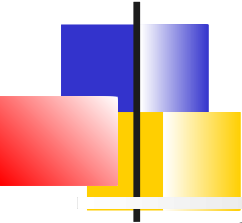


Injection par type

L'annotation **@Autowired** peut se placer à de nombreux endroits.

Déclaration d'attributs, arguments de constructeur, méthodes arbitraires...

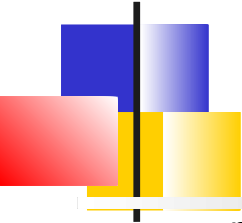
- Elle demande à Spring d'injecter un bean du type déclaré
- Généralement un seul bean est candidat à l'injection
- @Autowired a un attribut supplémentaire `required`, (true par défaut)



Examples

```
public class SimpleMovieLister {
    private MovieFinder movieFinder;
    @Autowired
    public void setMovieFinder(MovieFinder movieFinder) {
        this.movieFinder = movieFinder;
    }
}
...
public class MovieRecommender {
    private MovieCatalog movieCatalog;
    private CustomerPreferenceDao customerPreferenceDao;

    @Autowired
    public void prepare(MovieCatalog mCatalog, CustomerPreferenceDao cPD) {
        this.movieCatalog = mCatalog;
        this.customerPreferenceDao = cPD;
    }
    // ...
}
```



Examples (2)

```
public class MovieRecommender {
```

```
    @Autowired
```

```
    private MovieCatalog movieCatalog;
```

```
    private CustomerPreferenceDao customerPreferenceDao;
```

```
    @Autowired
```

```
    public MovieRecommender(CustomerPreferenceDao customerPreferenceDao) {
```

```
        this.customerPreferenceDao = customerPreferenceDao;
```

```
    }
```

```
    // ...
```

```
    }
```

```
    ...
```

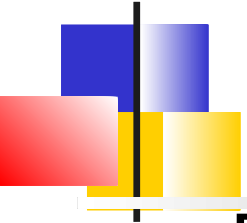
```
public class MovieRecommender {
```

```
    @Autowired
```

```
    private MovieCatalog[] movieCatalogs;
```

```
    // ...
```

```
    }
```

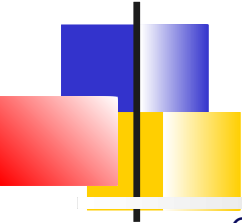


Injection implicite

Dans les dernières versions de Spring, l'annotation *@Autowired* n'est plus nécessaire lorsque l'on utilise l'injection par constructeur

- Si un attribut est déclaré comme ***final***, alors Spring sait qu'il faut l'initialiser dans le constructeur

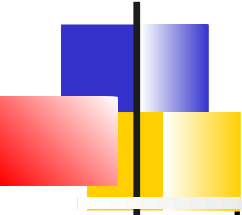
C'est ce qu'on appelle l'injection implicite, c'est une injection par type



Injection implicite

@Controller

```
public class MovieLister {  
    private final MovieDao movieDao ;  
    public MovieLister(MovieDao movieDao) {  
        this.movieDao = movieDao ;  
    }  
  
    public List<Movie> moviesDirectedBy(String arg) {  
        List<Movie> allMovies = movieDao.findAll();  
        List<Movie> ret = new ArrayList<Movie>() ;  
        for (Movie movie : allMovies ) {  
            if (!movie.getDirector().equals(arg))  
                ret.add(movie);  
        }  
        return ret;  
    }  
}
```



Exceptions dues à l'auto-wiring

L'injection par type peut provoquer des exceptions au démarrage de Spring.

- Cas 1 : Spring n'arrive pas à trouver de définitions de Beans correspondant au type :

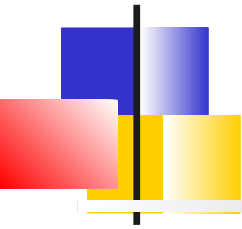
UnsatisfiedDependencyException, No qualifying bean of type "available:

expected at least 1 bean which qualifies as autowire candidate.

- Cas 2 : Spring trouve plusieurs Beans du type demandé

UnsatisfiedDependencyException, No qualifying bean of type "available:

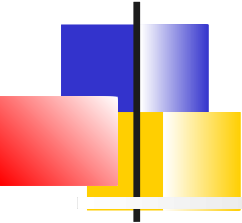
expected single matching bean but found 2.



@Resource et *@Qualifier*

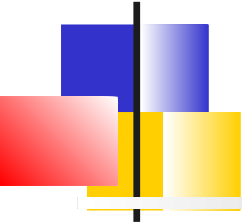
@Resource et ***@Qualifier*** permettent d'injecter un bean par son nom.

- Les annotations spécifient alors le nom du bean
- Si le nom n'est pas précisé, le nom du bean à injecter correspond au nom de la propriété



Exemples

```
public class MovieRecommender {  
    @Resource(name="myPreferenceDao")  
    private CustomerPreferenceDao cpDao;  
  
    // Le nom du bean recherché est "context"  
    @Resource  
    private ApplicationContext context;  
  
    // Plusieurs beans implémentent Formatter  
    @Autowired  
    @Qualifier("fooFormatter")  
    private Formatter formatter;  
    // ...  
}
```



Configuration via les annotations

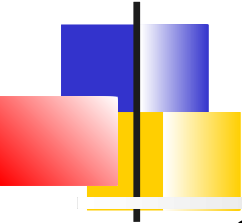
Classes de configuration

@Component et stéréotypes

Injection de dépendances

Propriétés de configuration

Environnement et profils

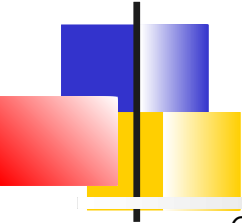


Introduction

Spring permet également d'injecter de simples valeurs (String, entiers, etc.) dans les propriétés des beans via des annotations :

- **@PropertySource** permet d'indiquer un fichier .properties permettant de charger des valeurs de configuration (clés/valeurs)
- **@Value** permet d'initialiser les propriétés des beans avec une expression **SpEL**¹ référençant une clé de configuration

Cela nécessite la présence d'un bean
PropertySourcesPlaceholderConfigurer



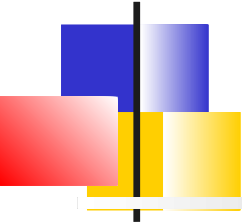
Exemple

```
@Configuration
@PropertySource("classpath:/com/myco/app.properties")
public class AppConfig {

    @Value("${my.property:0}")
    Integer myIntProperty ;

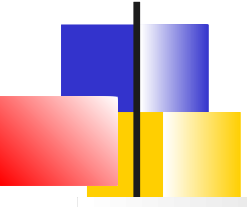
    @Autowired
    Environment env;

    @Bean
    public TestBean testBean() {
        TestBean testBean = new TestBean();
        testBean.setIntProperty(myIntProperty) ;
        testBean.setName(env.getProperty("testbean.name"));
        return testBean;
    }
}
```



Configuration via les annotations

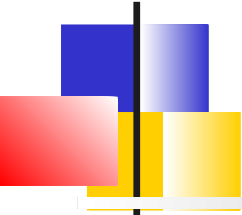
Classes de configuration
@Component et stéréotypes
Injection de dépendances
Propriétés de configuration
Environnement et profils



Environment

L'interface ***Environment*** est une abstraction modélisant 2 aspects :

- Les **propriétés** : Ce sont des propriétés de configuration des beans. Ils proviennent des fichier .properties, d'argument de commande en ligne ou autre ...
- Les **profils** : Groupes nommés de Beans, les beans sont enregistrés seulement si le profil est activé au démarrage

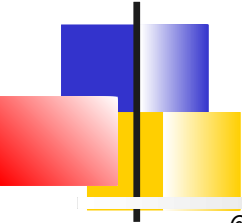


Annotations utilisant les profils

Tout `@Component` ou `@Configuration` peut être marqué avec **`@Profile`** pour limiter son chargement

```
@Configuration
@Profile("production")
public class ProductionConfiguration {
    // ...
}

@Service
@Profile("kafka")
public class MyKafkaServiceImpl implements MyService {
    // ...
}
```

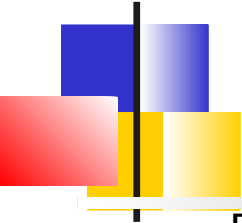


Exemple

Bd de dév et BD de prod

```
@Configuration
@Profile("development")
public class StandaloneDataConfig {
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.HSQL)
            .addScript("classpath:com/bank/config/sql/schema.sql")
            .addScript("classpath:com/bank/config/sql/test-data.sql")
            .build();
    }
}

@Configuration
@Profile("production")
public class JndiDataConfig {
    @Bean(destroyMethod="")
    public DataSource dataSource() throws Exception {
        Context ctx = new InitialContext();
        return (DataSource) ctx.lookup("java:comp/env/jdbc/datasource");
    }
}
```

Activation d'un profil

Programmatically :

```
AnnotationConfigApplicationContext ctx = new  
AnnotationConfigApplicationContext();  
ctx.getEnvironment().setActiveProfiles("development");  
ctx.register(SomeConfig.class, StandaloneDataConfig.class,  
JndiDataConfig.class);  
ctx.refresh();
```

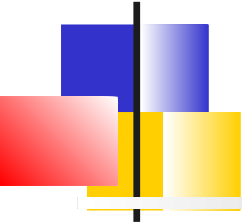
Ligne de commande :

Propriété Java

```
java -jar myjar -Dspring.profiles.active="profile1,profile2"
```

Argument SpringBoot

```
java -jar myjar --spring.profiles.active="profile1"
```



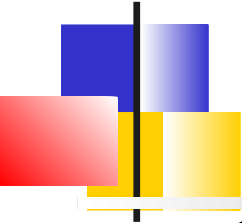
SpringBoot

L'auto-configuration

Starters SpringBoot

Développer avec SpringBoot

Propriétés de configuration



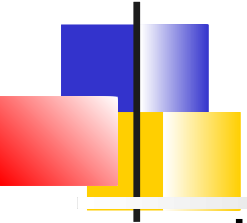
Introduction

Spring Boot a été conçu pour simplifier le démarrage
et le développement de nouvelles applications
Spring

Ne nécessite aucune configuration préalable :

Dès la première ligne de code, on a une application
fonctionnelle

=> Offrir une expérience de développement
simplifiant à l'extrême l'utilisation des technologies
existantes



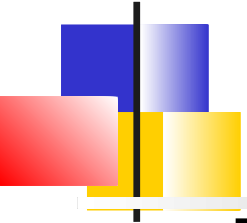
Auto-configuration

Le concept principal de SpringBoot est **l'auto-configuration**

SpringBoot est capable de détecter la nature de l'application et de configurer les beans Spring nécessaires

- Permet de démarrer rapidement avec une configuration par défaut puis de graduellement surcharger la configuration pour ses besoins spécifiques

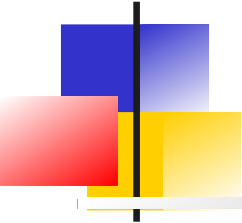
Les mécanismes sont différents en fonction du langage : Groovy, Java ou Kotlin



Auto-configuration (Java)

En fonction des librairies présentes au moment de l'exécution, Spring Boot crée tous les beans techniques nécessaires avec une configuration par défaut.

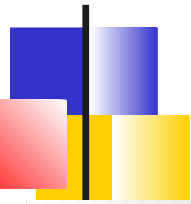
- Par exemple, si il s'aperçoit que des librairies Web sont présentes, il démarre un serveur Tomcat embarqué sur le port 8080 et applique la configuration par défaut de Spring MVC
- Si il s'aperçoit que le driver H2 est dans le classpath, il crée automatiquement un pool de connexions vers la base
- Etc...



Personnalisation de la configuration

La configuration par défaut peut être surchargée par différent moyens

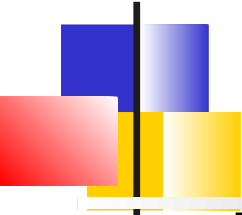
- Les propriétés qui modifient les valeurs par défaut des beans techniques via :
 - Des variables d'environnement
 - Des arguments de la ligne de commande
 - Des fichiers de configuration externe (.property ou .yml).
Différents fichiers peuvent être activés en fonction de profils
- Avec des classes de configuration redéfinissant les beans par défaut
- En utilisant des classes spécifiques du framework (exemple classes *Configurer)



Autres apports de SpringBoot

En plus de l'auto-configuration, SpringBoot offre d'autres bénéfices pour les développeurs :

- Simplification de la gestion des dépendances vers les librairies OpenSource via les **starters**
- Assistant de création de projet avec ***SpringInitializer***
- Point central de la configuration des propriétés avec ***application.properties/yml***
- Plugins pour les **IDEs** (SpringTools suite ou autre)
- Plugins pour les **outils de build** (Gradle et Maven)



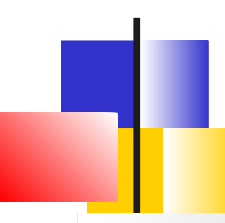
Gestion des dépendances

La gestion des dépendances est simplifiée grâce aux starters.

Ce sont des groupes de dépendances permettant d'intégrer une technologie ou apportant une fonctionnalité qui sont déclarés dans le fichier de build

=> Pour les développeurs, ce mécanisme simplifie à l'extrême la gestion des versions des dépendances.

Plus qu'un seul numéro de version à gérer : Celui de SpringBoot



<https://start.spring.io/>



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy

☒ Maven

Spring Boot

☐ 4.0.1 (SNAPSHOT) ☒ 4.0.0 ☐ 3.5.9 (SNAPSHOT) ☐ 3.5.8

☐ 3.4.13 (SNAPSHOT) ☐ 3.4.12

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

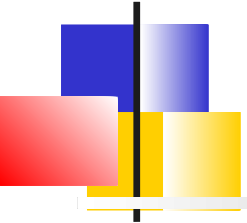
Configuration ☒ Properties ☐ YAML

Java ☐ 25 ☐ 21 ☒ 17

Dependencies

[ADD DEPENDENCIES...](#) CTRL + B

No dependency selected



Fichiers générés par l'assistant

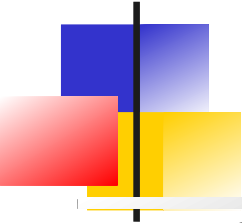
.gitignore, Help.md

Scripts de build (mvnw ou gradlew)

Classe de démarrage de SpringBoot
(src/main/java)

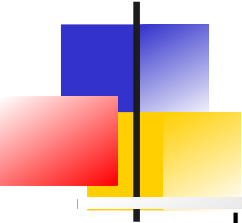
Classe de test de la configuration
(src/test/java)

Fichier de configuration des propriétés
(src/main/resources)



Exemple Gradle

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.1.2'  
    id 'io.spring.dependency-management' version '1.1.2'  
}  
group = 'org.formation'  
version = '0.0.1-SNAPSHOT'  
java {  
    sourceCompatibility = '17'  
}  
repositories { mavenCentral() }  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-webflux'  
    implementation 'org.springframework.kafka:spring-kafka'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
    testImplementation 'io.projectreactor:reactor-test'  
    testImplementation 'org.springframework.kafka:spring-kafka-test'  
}  
tasks.named('test') { useJUnitPlatform() }
```



Plug-in Maven/Gradle

L'initializer crée des scripts (mvnw ou gradlew) pour les environnements Linux et Windows.

- Ce sont des wrappers de l'outil de build garantissant que tous les développeurs utilisent la même version de l'outil de build.

La commande la + importante dans un contexte Maven :

./mvnw clean package

=> Génère un fat-jar

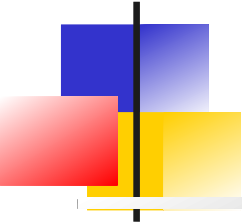
L'application peut alors être démarrée en ligne de commande par :

java -jar target/myAppli.jar

Avec gradle :

./gradlew build

java -jar build/libs/mymodule-0.0.1-SNAPSHOT.jar



Autres tâches de build

Packager une image OCI :
spring-boot:build-image

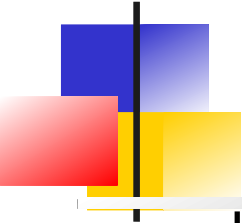
Exécuter l'application
spring-boot:run

Exécuter avec le classpath de tes
spring-boot:test-run

Construire une image native
package -Pnative

Exécuter des tests d'intégration
spring-boot:start
spring-boot:stop

Renseigner le endpoint /actuator/info avec les informations de build
spring-boot:build-info

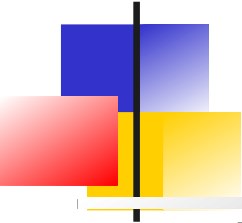


Classe de démarrage

La classe de démarrage est annotée via `@SpringBootApplication`, annotation qui englobe :

- **`@Configuration`** : Permet de définir des méthodes `@Bean`
- **`@ComponentScan`** : Scan des annotations dans les sous-packages
- **`@EnableAutoConfiguration`** : Activation du mécanisme d'auto-configuration de SpringBoot

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        // Création du contexte Spring
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

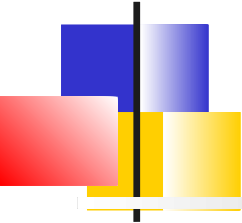


Classe de test

La classe de test est annotée via
@SpringBootTest :

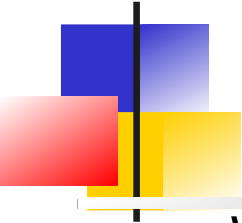
- Permet de créer le contexte Spring avant l'exécution du test junit5

```
@SpringBootTest
class DemoApplicationTests {
    @Test
    void contextLoads() {
        // Si le test passe, la configuration SpringBoot est OK
    }
}
```



SpringBoot

L'auto-configuration
Starters SpringBoot
Développer avec SpringBoot
Propriétés de configuration



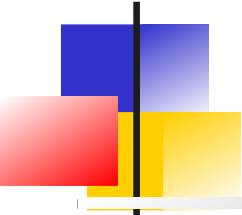
Starters les + importants

Web

- *-**web** : Application web ou API REST. Framework Spring MVC
- *-**reactive-web** : Application web ou API REST en mode réactif. Framework Spring WebFlux
- *-**actuator** : Observabilité via endpoints HTTP
- *-**validation** : Contraintes de validation sur les classes

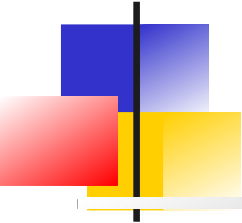
Cœurs :

- *-**logging** : Utilisation de logback (Tjs présent)
- *-**test** : Test avec Junit, AssertJ et Mockito (Tjs présent)



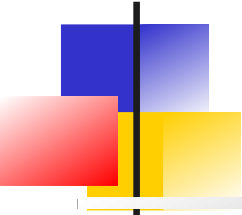
Starters développement

- *-**devtools** : Fonctionnalités pour le développement
- *-**lombok** : Simplification du code Java
- *-**configuration-processor** : Complétion des propriétés de configuration applicatives disponibles dans l'IDE
- *-**docker-compose** : Support pour démarrer les services de support via docker-compose (BD, Kafka, etc..)
- *-**graalvm-native** : Support pour construire des images natives
- *-**modulith** : Support pour construire des applications monolithes modulaires



Sécurité

- *-**security** : Spring Security, sécurisation des URLs et des services métier
- *-**oauth2-client** : Pour obtenir un jeton OAuth d'un serveur d'autorisation
- *-**oauth2-resource-server** : Sécurisation des URLs via OAuth
- *-**oauth2-authorization-server** : Serveur d'autorisation
- *-**web-authn** : Support de WebAuthn



Starters SQL

jdbc : JDBC avec pool de connexions Tomcat

Spring Data JPA : Spring Data avec Hibernate et JPA

Spring Data JDBC : Spring Data avec jdbc

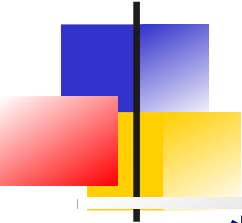
Spring Data R2DBC : Spring Data avec jdbc reactif

LiquiBase Migration : Migration de schéma avec Liquibase

Flyway Migration : Migration de schéma avec Flyway

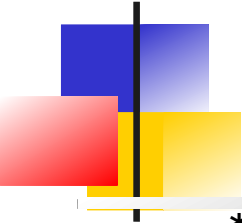
JOOQ Access Layer : API fluent pour construire des requêtes SQL

****-<drivers>*** : Accès aux driver JDBC (MySQL, Postgres, H2, HyperSonic, DB2)



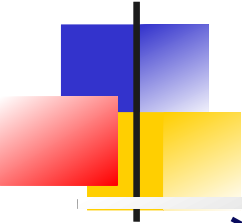
Starters NOSQL

- *-***data-cassandra***, *-***data-reactive-cassandra*** : Base distribuée Cassandra
- *-***data-neo4j*** : Base de données orienté graphe de Neo4j
- *-***data-couchbase*** *-***data-reactive-couchbase*** : Base NoSQL CouchBase
- *-***data-redis*** *-***data-reactive-redis*** : Base NoSQL Redis
- *-***data-geode*** : Stockage de données via Geode
- *-***data-elasticsearch*** : Base documentaire indexée ElasticSearch
- *-***data-mongodb*** *-***data-reactive-mongodb*** : Base NoSQL MongoDB



Messaging

- *-**integration**: Spring Integration (Abstraction de + haut niveau pour implémenter des patterns d'intégration de façon déclaratif)
- *-**kafka**: Intégration avec Apache Kafka
- *-**kafka-stream**: Intégration avec l'API KafkaStream
- *-**rabbitmq**: Intégration avec Rabbit MQ
- *-**activemq5** : ActiveMQ avec JMS
- *-**artemis** : ApacheMQ avec Artemis
- *-**pulsar**, *-**reactive-pulsar** : Messagerie PULSAR
- *-**websocket** : Servlet avec STOMP et SockJS
- *-**rsocket** : SpringMessaging et Netty
- *-**camel** : Intégration avec Apache Camel
- *-**solacePubSub** : Intégration avec Solace



Quelques starters IA

***-openai**: OpenAI

***-azure-ai, *-azure-search**: Intégration avec Apache Kafka

-amazon-bedrocks-: Amazon bedrocks

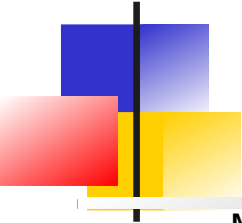
-google-genai-: Google genAI

***-mistral-ai** : Mistral AI

Bd vectorielle : Cassandra, Chroma, Couchbase, elasticsearch, gemfire, Mongo, Neo4j

MCP : MCP client et serveur

...



Autres Starters Web

Moteur de templates HTML

*-**thymeleaf** : Spring MVC avec des vues Thymeleaf

*-**mustache** : Spring MVC avec Mustache

*-**groovy-templates** : Spring MVC avec gabarits Groovy

*-**freemarker** : Spring MVC avec freemarker

Autres

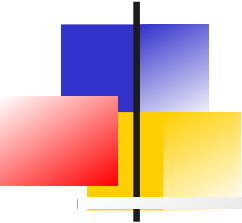
*-**graphql** : API GraphQL

*-**rest-repository**, **restrepository-explorer**, *-**hateoas** : Génération API Rest à partir des repositories de Spring Data

*-**jersey** : API Restful avec JAX-RS et Jersey

*-**webservices** : Services SOAP

*-**vaadin**, *-**hilla** : Framework pour applis web



Autres Starters

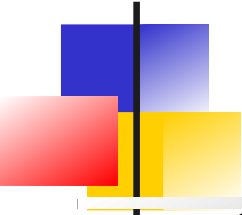
*-**batch** : Gestion de batchs

*-**mail** : Envois de mails

*-**cache** : Support pour un cache

*-**quartz** : Intégration avec Scheduler

*-**shell** : Support pour commandes en ligne



Spring Cloud

Services cloud : Facilité de déploiement

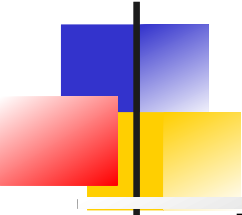
Amazon, Google Cloud, Azure, Cloud Foundry,
Alibaba

Architecture Micro-services

Services de discovery, de configuration externalisée,
de répartition de charge, de gateway, de résilience

Spring Cloud Contract : Génération de tests et mock
servers

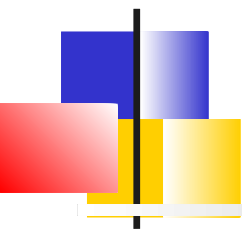
Service de monitoring, de tracing, etc ...



Observabilité

Depuis la version 3.x, SpringBoot s'appuie fortement sur Micrometer.

- Des starters permettent de publier les métriques micrometer vers des système de visualisation :
DataDog, Dynatrace, Influx, Graphite, New Relic, Prometheus, Wavefront
- D'autres starters permettent la tracabilité des requêtes dans les architecture micro-services :
OpenTelemetry

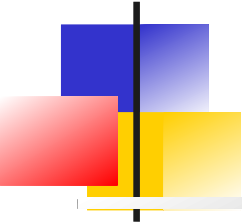


SpringBoot

L'auto-configuration
Starters SpringBoot

Développer avec SpringBoot

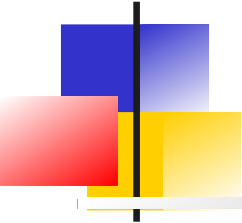
Propriétés de configuration



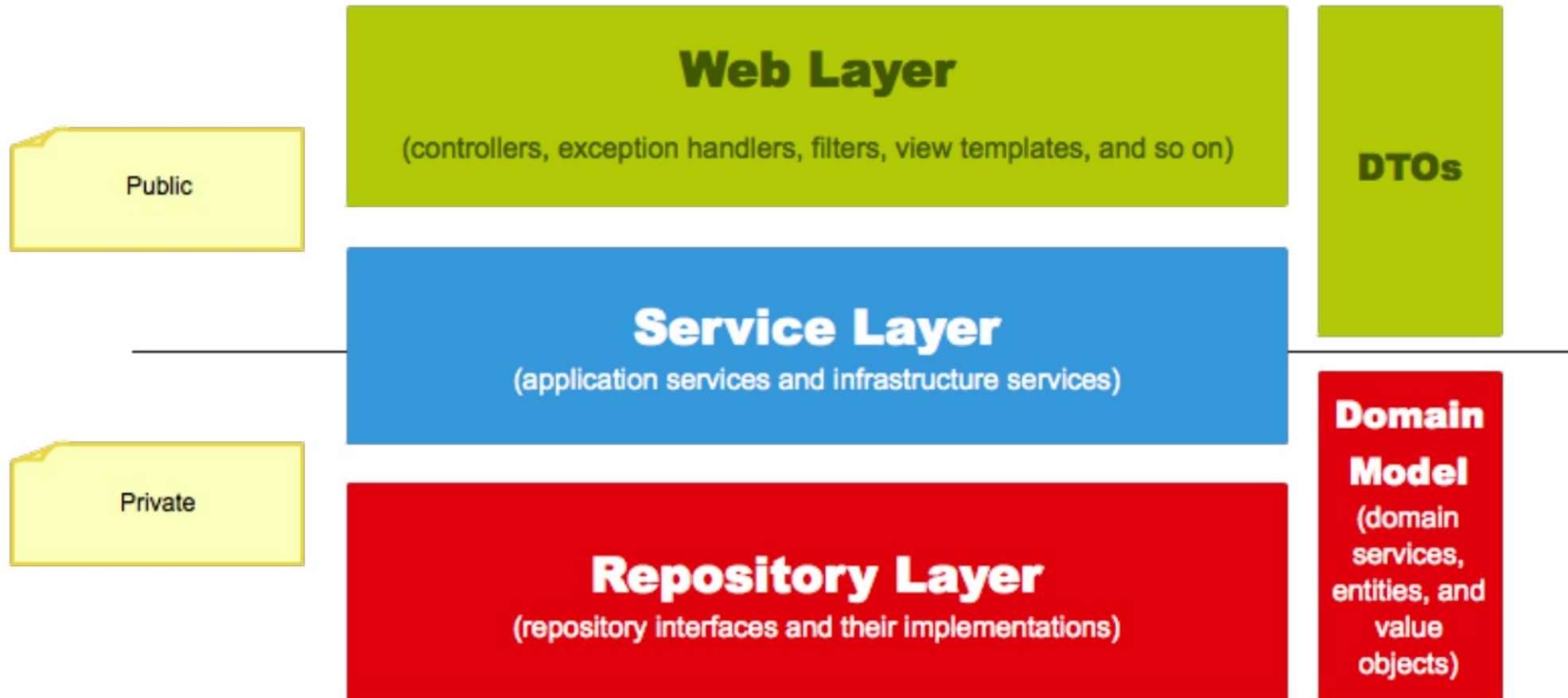
Structure projet

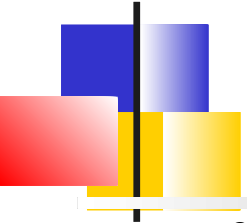
Aucune obligation mais des recommandations :

- Placer la classe Main dans le package racine
- L'annoter avec **@SpringBootApplication**, équivalent à :
 - @EnableAutoConfiguration
 - @ComponentScan
 - @Configuration



Architecture classique

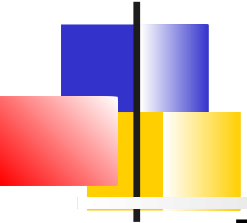




Structure typique

com

```
+ - example
  +- myproject
    +- Application.java
    +- OneConfig.java
    |
    +- domain
      +- Customer.java
      +- CustomerRepository.java
    |
    +- service
      +- CustomerService.java
    |
    +- web
      +- CustomerController.java
```



Spring Tool Suite

Distribué par Pivotal pour VSCode, Atom ou Eclipse

L'intégration existe avec d'autres IDE

Apports :

- **Assistant de création de projet** connecté directement à Spring Initializr
- **Auto-completion** des propriétés de configuration
- **Boot Dashboard** pour démarrer les applications
- **Run configurations** : configurations de démarrage

New Spring Starter Project



Name:

Type: Packaging:

Java Version: Language:

Boot Version:

Group:

Artifact:

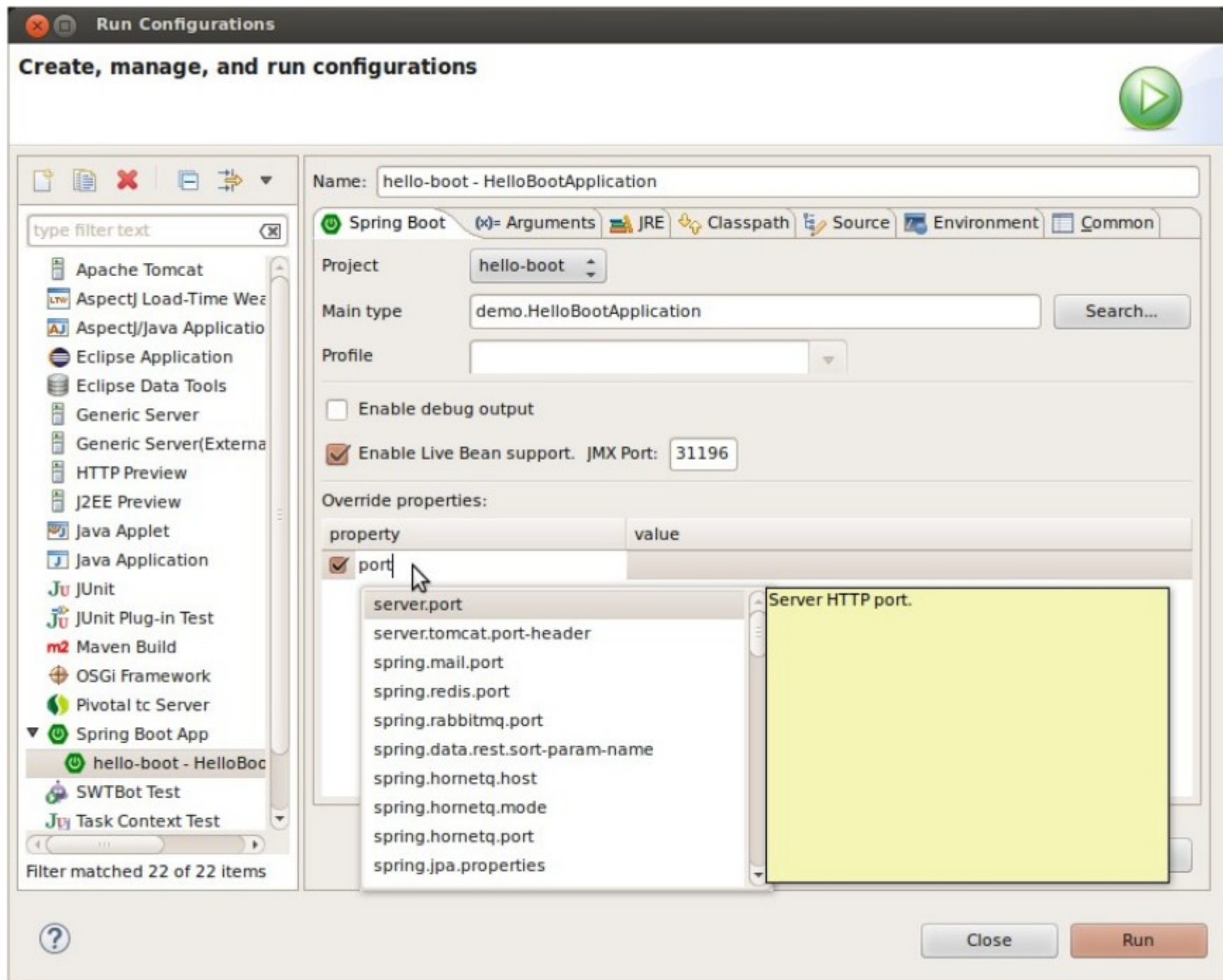
Version:

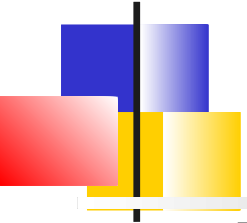
Description:

Package:

Dependencies

- | | | | |
|---|-----------------------------------|---|---|
| ☐ AMQP | ☐ AOP | ☐ Actuator | ☐ Apache Derby |
| ☐ Atomikos (JTA) | ☐ Batch | ☐ Bitronix (JTA) | ☐ Cloud Connectors |
| ☐ Elasticsearch | ☐ Facebook | ☐ Freemarker | ☐ Gemfire |
| ☐ Groovy Templates | ☐ H2 | ☐ HATEOAS | ☐ HSQLDB |
| ☐ Integration | ☐ JDBC | ☐ JMS | ☐ JPA |
| ☐ Jersey (JAX-RS) | ☐ LinkedIn | ☐ Mail | ☐ Mobile |
| ☐ MongoDB | ☐ Mustache | ☐ MySQL | ☐ Redis |

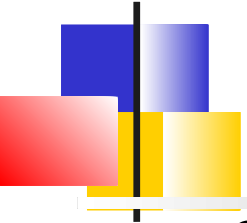




Rechargement de code

Étant donné que les applications Spring Boot sont de simples applications Java, le rechargement de code à chaud doit être pris en charge.

=> Cela élimine le besoin de redémarrer l'application à chaque changement de code

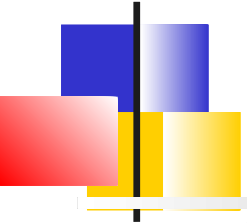


Dev Tools

spring-boot-devtools starter peut être ajouté

Cela permet :

- Ajout de propriétés de configuration pour le développement.
Ex : `spring.thymeleaf.cache=false`
- Rechargement de contexte automatique lorsqu'une classe ou la configuration change.
- LiveReload Server : autoriser le rechargement du navigateur pour le développement Web

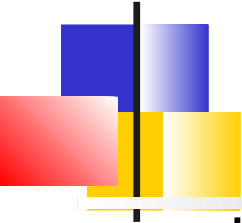


Logging

Spring utilise Common Logging en interne mais permet de choisir son implémentation

Des configurations sont fournies pour :

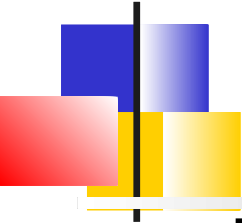
- Java Util Logging
- Log4j2
- Logback (default)



Format des traces

Une ligne contient :

- Timestamp à la ms
- Severity level : ERROR, WARN, INFO, DEBUG ou TRACE.
- Process ID
- Un séparateur ---
- Nom de la thread entouré de [].
- Nom du Logger <=> Nom de la classe
- Un message
- Une note entouré de []



Configurer les traces

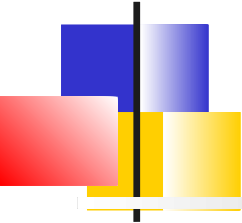
Par défaut, Spring affiche les messages de niveau ERREUR, WARN et INFO sur la console

- `java -jar myapp.jar -debug` : Active les messages de DEBUG
- Propriétés `logging.file` et `logging.path` pour générer un fichier de log
- Les niveaux de gravité peuvent être modifiés pour chaque logger

```
logging.level.root=WARN
```

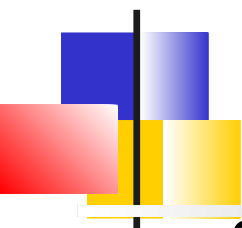
```
logging.level.org.springframework.web=DEBUG
```

```
logging.level.org.hibernate=ERROR
```



SpringBoot

L'auto-configuration
Starters SpringBoot
Développer avec SpringBoot
Propriétés de configuration



Propriétés de configuration

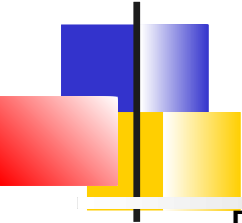
Spring Boot permet d'externaliser la configuration des beans :

- Ex : Externaliser l'adresse de la BD, la configuration d'un client, ...

On peut utiliser des **fichiers properties ou YAML**, des variables d'environnement ou des arguments de commande en ligne.

Les valeurs des propriétés sont ensuite injectées dans les beans :

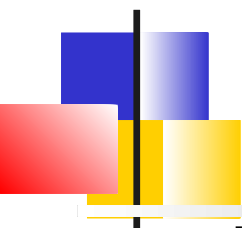
- Directement via l'annotation **@Value**
- Ou associer à un objet structuré via l'annotation **@ConfigurationProperties**



Priorités

De nombreux niveaux de propriétés différents mais en résumé l'ordre des propriétés est :

1. spring-boot-devtools.properties si devtools est activé (SpringBoot)
2. Les propriétés de test
- 3. La ligne de commande. Ex : --server.port=9000**
4. Environnement REST, Servlet, JNDI, JVM
- 5. Variables d'environnement de l'OS. Ex : export SERVER_PORT=8000**
6. Propriétés avec des valeurs aléatoires
7. Propriétés spécifiques à un profil
- 8. application.properties , yml**
9. Annotation @PropertySource dans la configuration
10. Les propriétés par défaut spécifié par SpringApplication.setDefaultProperties

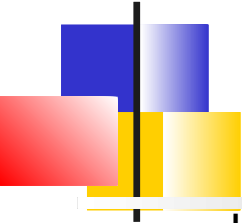


application.properties (.yml)

Les fichiers de propriétés (application.properties/.yml) sont généralement placés dans les emplacements suivants :

- Un sous-répertoire config
- Le répertoire courant
- Un package config dans le classpath
- A la racine du classpath

En respectant ces emplacements standards, SpringBoot les trouve tout seul



Valeur filtrée

Les fichiers supportent les valeurs filtrées.

```
app.name=MyApp
```

```
app.description=${app.name} is a Boot app.
```

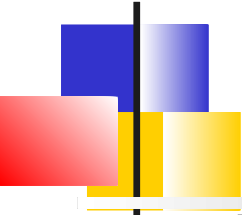
Les valeurs aléatoires :

```
my.secret=${random.value}
```

```
my.number=${random.int}
```

```
my.bignumber=${random.long}
```

```
my.uuid=${random.uuid}
```

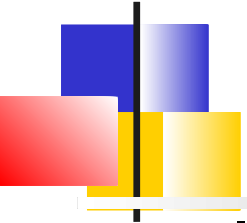


Injection de propriété : *@Value*

La première façon de lire une valeur configurée est d'utiliser l'annotation ***@Value***.

```
@Value("${my.property}")  
private Integer myProperty ;
```

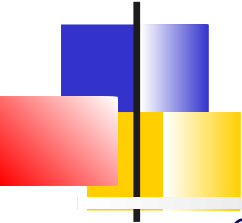
Dans ce cas, aucun contrôle n'est effectué sur la valeur effective de la propriété



Valider la configuration

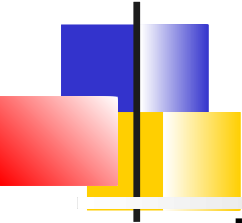
Il est possible de forcer la validation des propriétés de configuration à l'initialisation du conteneur.

- Utiliser une classe annotée par **@ConfigurationProperties** et **@Validated**
- Positionner des contraintes de **javax.validation** sur les attributs de la classe



Exemple

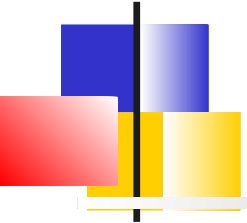
```
@Component  
@ConfigurationProperties("app")  
@Validated  
public class MyAppProperties {  
  
    @Pattern(regex = "\\d{3}-\\d{3}-\\d{4}")  
    private String adminContactNumber;  
  
    @Min(1)  
    private int refreshRate;  
  
    .....  
}
```



Propriétés spécifiques à un profil

Les propriétés spécifiques à un profil (ex : intégration, production) sont spécifiées différemment en fonction du format properties ou .yaml.

- Si l'on utilise le format .properties, on peut fournir des fichiers complémentaires :
application-{profile}.properties
- Si l'on utilise le format .yaml tout peut se faire dans le même fichier



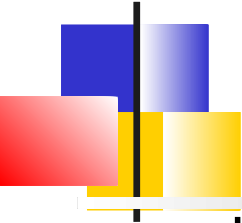
Exemple fichier .yaml

```
server:  
  address: 192.168.1.100
```

```
---
```

```
spring:  
  config:  
    activate:  
      on-profile:  
        -prod
```

```
server:  
  address: 192.168.1.120
```

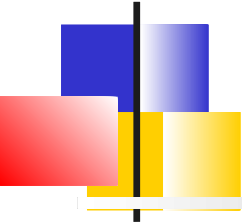


Activation des profils

Les profils sont activés généralement par la propriété ***spring.profiles.active*** qui peut être positionnée :

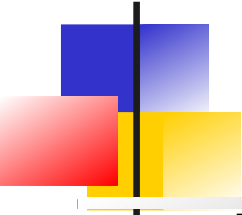
- Dans un fichier de configuration
- En commande en ligne via :
--spring.profiles.active=dev,hsqldb
- Programmatically, via :
SpringApplication.setAdditionalProfiles(...)

Plusieurs profils peuvent être activés simultanément



Persistence

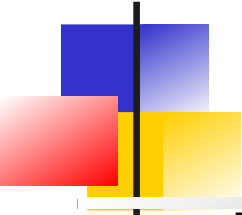
Principes de SpringData SpringData JPA



Introduction

La mission de Spring Data est de fournir un modèle de programmation simple et cohérent pour l'accès aux données quelque soit la technologie sous-jacente (Relationnelle, NoSQL, Cloud, Moteur de recherche)

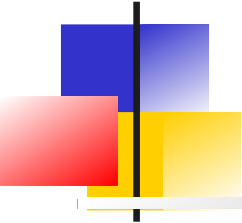
Spring Data est donc le projet qui encadre de nombreux sous-projets spécialisés sur une API de persistance (jdbc, JPA, Mongo, ...)



Apports de SpringData

Les apports sont :

- Une abstraction de la notion de repository et de mapping objet
- La génération dynamique de requêtes basée sur des règles de nommage des méthodes
- Des classes d'implémentations de bases pouvant être injectées : *Template.java
- Un support pour l'audit (Date de création, dernier changement)
- La possibilité d'intégrer du code spécifique au repository



Interface Repository

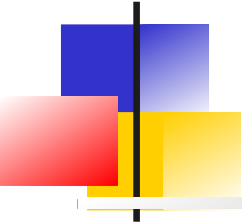
L'interface centrale de Spring Data est **Repository** (C'est une classe marqueur)

L'interface prend en arguments de type

- la classe persistante du domaine
- son id.

La sous-interface **CrudRepository** ajoute les méthodes CRUD

Des abstractions spécifiques aux technologies sont également disponibles *JpaRepository*, *MongoRepository*, ...



Interface CrudRepository

```
public interface CrudRepository<T, ID extends  
    Serializable> extends Repository<T, ID> {
```

```
    <S extends T> S save(S entity);
```

```
    T findOne(ID primaryKey);
```

```
    Iterable<T> findAll();
```

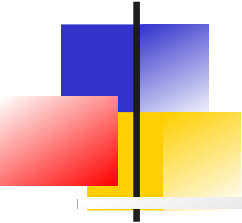
```
    Long count();
```

```
    void delete(T entity);
```

```
    boolean exists(ID primaryKey);
```

```
    // ... et d'autres.
```

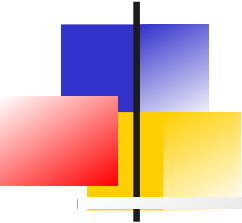
```
}
```



Exemple

```
public interface MemberRepository extends  
    CrudRepository<Member, Long> {}
```

Juste avec cette définition d'interface,
Spring créera une implémentation qui
possède déjà toutes les méthodes
CRUD de CrudRepository

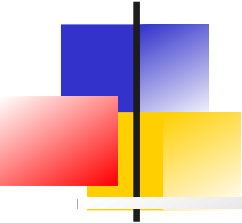


Requêtes additionnelles

Après avoir étendu l'interface, il est possible de définir des méthodes permettant d'effectuer des requêtes spécifiques

Spring déduit les requêtes à effectuer :

- Soit à partir du nom de la méthode
- Soit à partir de l'annotation **@Query**



Méthodes de requête

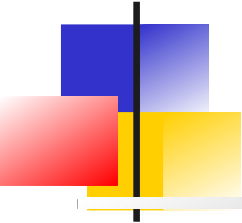
Lors de l'utilisation du nom de la méthode, celles-ci doivent être préfixées comme suit :

- Recherche : ***find*By****
- Comptage : ***count*By****
- Suppression : ***delete*By****
- Récupération : ***get*By****

La première ******* peut indiquer un flag (comme Distinct par exemple)

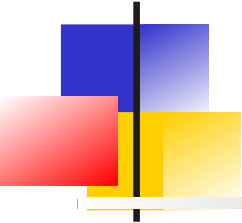
Le terme ***By*** marque la fin de l'identification du type de requête

Le reste est parsé et spécifie la clause where et éventuellement ***orderBy***



Exemple

```
public interface MemberRepository
extends JpaRepository<Member, Long> {
    /**
     * Tous les membres ayant un email particulier.
     * @param email
     * @return
     */
    public List<Member> findByEmail(String email);
    /**
     * Chargement de la jointure one2Many.
     * @param id
     * @return
     */
    @Query("from Member m left join fetch m.documents where m.id =:id")
    public Optional<Member> fullLoad(Long id);
}
```



Clause Where

Les noms des méthodes consistent généralement de propriétés de l'entité combinées avec AND et OR

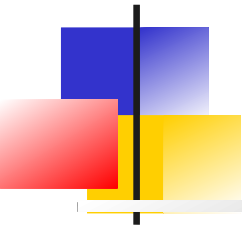
Des opérateurs peuvent également être précisés :
Between, LessThan, GreaterThan, Like

Le flag *IgnoreCase* peut être attribué individuellement aux propriétés ou de façon globale

```
findByLastnameIgnoreCase(...)
```

```
findByLastnameAndFirstnameAllIgnoreCase(...)
```

La clause *order* de la requête peut être précisée en ajoutant `OrderBy(Asc/Desc)` à la fin de la méthode



Gestion des paramètres

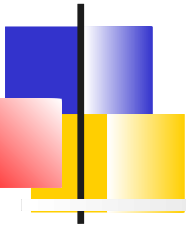
En plus des paramètres de la requête, SpringBoot est capable de reconnaître les arguments de types **Pageable** ou **Sort** pour appliquer la pagination et le tri dynamiquement

Les valeurs de retours peuvent alors être :

- *Optional<entity>* , *List<entity>*
- Page connaît le nombre total d'éléments en effectuant une requête *count*,
- Slice ne sait que si il y a une page suivante

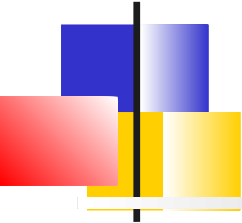
Exemples :

```
Page<User> findByLastname(String lastname, Pageable pageable);  
Slice<User> findByLastname(String lastname, Pageable pageable);  
List<User> findByLastname(String lastname, Sort sort);  
List<User> findByLastname(String lastname, Pageable pageable);
```



Mots-clés supportés pour JPA

And, Or, Is, Equals, Between,
LessThan, LessThanEqual,
GreaterThan, GreaterThanEqual,
After, Before, IsNull, IsNotNull,
NotNull, Like, NotLike,
StartingWith, EndingWith,
Containing, OrderBy, Not, In,
NotIn, True, False, IgnoreCase



Utilisation de *@Query*

@Query utilise le langage d'interrogation, (ex JPQL), pour spécifier la requête:

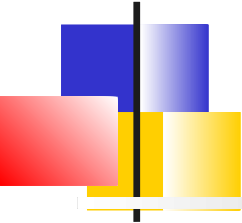
- Méthode la plus prioritaire
- A l'avantage de se situer sur la classe Repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    @Query("select u from User u where u.emailAddress = ?1")  
    User findByEmailAddress(String emailAddress);
```

```
    @Query("select u from User u where u.firstname like %?1")  
    List<User> findByFirstnameEndsWith(String firstname);
```

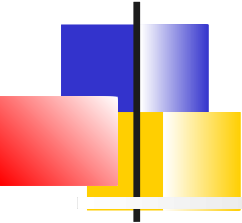
```
    @Query("select u.id, LENGTH(u.firstname) as fn_len from User u where u.lastname like ?1%")  
    List<Object[]> findByAsArrayAndSort(String lastname, Sort sort);
```

```
    @Query("select u from User u where u.firstname = :firstname or u.lastname = :lastname")  
    User findByLastnameOrFirstname(@Param("lastname") String lastname, @Param("firstname")  
        String firstname);  
}
```



Persistence

Principes de SpringData
SpringData JPA



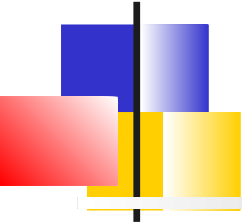
Apports

spring-boot-starter-data-jpa fournit les dépendances suivantes :

- Hibernate
- Spring Data JPA
- Spring ORMs

Par défaut, toutes les classes annotée par *@Entity*, *@Embeddable* ou *@MappedSuperclass* sont scannées et prises en compte

L'emplacement de départ du scan peut être réduit avec ***@EntityScan***

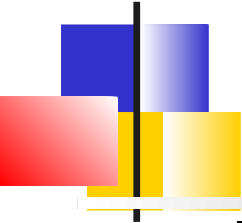


Rappels : Classes entités et associations

```
@Entity
public class Theme {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String label;
    @OneToMany(cascade = CascadeType.ALL)
    private Set<MotClef> motclefs = new HashSet<MotClef>();
}
```

```
@Entity
public class MotClef {
    @Id
    private Long id;
    private String mot;

    public MotClef(){}
}
```

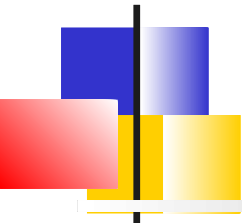


Configuration source de données / Rappels

Pour accéder à une BD relationnelle, Java utilise la notion de ***DataSource*** (interface représentant un pool de connections BD)

Une data source se configure via :

- Une URL JDBC
- Un compte base de donnée
- Un driver JDBC
- Des paramètres de dimensionnement du pool

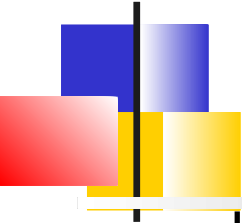


Support pour une base embarquée

Spring Boot peut configurer automatiquement les bases de données H2, HSQL et Derby.

Il n'est pas nécessaire de fournir d'URL de connexion, les dépendances suffisent :

```
implementation org.springframework.boot :spring-boot-starter-data-jpa  
runtime org.hsqldb :hsqldb
```



Base de production

Les bases de production peuvent également être auto-configurées.

Les propriétés requises à configurer sont :

`spring.datasource.url=jdbc:mysql://localhost/test`

`spring.datasource.username=dbuser`

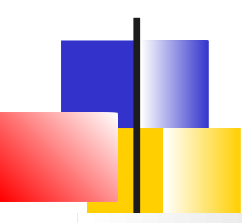
`spring.datasource.password=dbpass`

Voir *DataSourceProperties* pour l'ensemble des propriétés disponibles

L'implémentation du pool sous-jacent est Hikari

Cela peut être surchargée par la propriété :

`spring.datasource.type`



Configuration du pool

Des propriétés sont également spécifiques à l'implémentation de pool utilisée.

Par exemple pour Hikari :

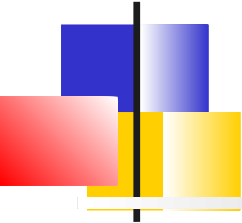
Timeout en ms si pas de connexions dispo.

```
spring.datasource.hikari.connection-timeout=10000
```

Dimensionnement du pool

```
spring.datasource.hikari.maximum-pool-size=50
```

```
spring.datasource.hikari.minimum-idle= 10
```



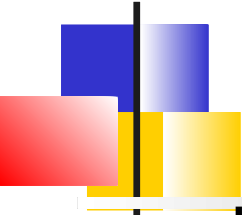
Mode *create*, *create-drop*

Hibernate peut être configurée en mode ***create***, ***create-drop***

La base est recréé à chaque démarrage et il est possible d'exécuter un script d'initialisation des données (Par défaut *import.sql* à la racine du classpath)

=> La base est dans un état connu à chaque démarrage.

Idéal pour le dév et le test



Propriétés

Les bases de données JPA embarquées sont créées automatiquement.

Pour les autres, il faut préciser la propriété

spring.jpa.hibernate.ddl-auto

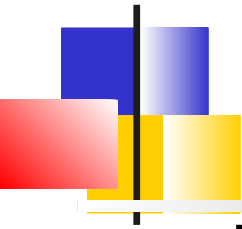
- 5 valeurs possibles : *none, validate, update, create, create-drop*

Ou utiliser les propriétés natives d'Hibernate, spécifiées en utilisant le préfixe

spring.jpa.properties.*

Ex :

```
spring.jpa.properties.hibernate.globally_quoted_identifiers=true
```

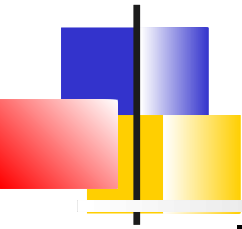



Comportement transactionnel des Repository

Par défaut, les méthodes CRUD sont transactionnelles.

Pour les opérations de lecture, l'indicateur *readOnly* de configuration de transaction est positionné.

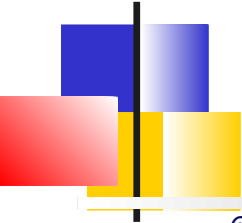
Toutes les autres méthodes sont configurées avec un *@Transactional* simple afin que la configuration de transaction par défaut s'applique



@Transactional et *@Service*

Il est courant d'utiliser une façade (bean *@Service*) pour implémenter une fonctionnalité métier nécessitant plusieurs appels à différents Repositories

L'annotation ***@Transactional*** permet alors de délimiter une transaction pour des opérations non CRUD.



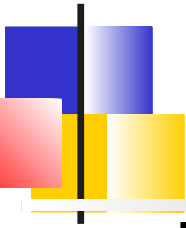
Example

@Service

```
class UserManagementImpl implements UserManagement {  
    private final UserRepository userRepository;  
    private final RoleRepository roleRepository;  
    public UserManagementImpl(UserRepository userRepository, RoleRepository  
        roleRepository) {  
        this.userRepository = userRepository;  
        this.roleRepository = roleRepository;  
    }  
}
```

@Transactional

```
public void addRoleToAllUsers(String roleName) {  
    Role role = roleRepository.findByName(roleName);  
    for (User user : userRepository.findAll()) {  
        user.addRole(role);  
        userRepository.save(user);  
    }  
}
```



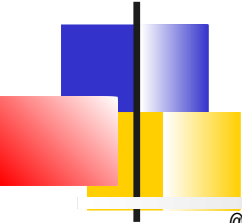
Configuration des Templates

Les beans ***JdbcTemplate*** et ***NamedParameterJdbcTemplate*** sont auto-configurés et peuvent donc être directement injectés

Leur comportement peut être personnalisé par les propriétés

spring.jdbc.template.*

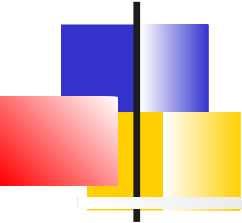
Ex : *spring.jdbc.template.max-rows=500*



Example Template

@Repository

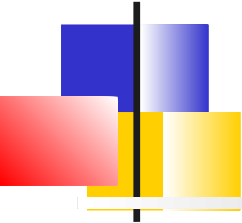
```
public class UserDaoImpl implements UserDao {  
    private final String INSERT_SQL = "INSERT INTO USERS(name, address, email) values(:name,:email)";  
    private final String FETCH_SQL_BY_ID = "select * from users where record_id = :id";  
  
    @Autowired  
    private NamedParameterJdbcTemplate namedParameterJdbcTemplate;  
  
    public User create(final User user) {  
        KeyHolder holder = new GeneratedKeyHolder();  
        SqlParameterSource parameters = new MapSqlParameterSource()  
            .addValue("name", user.getName())  
            .addValue("email", user.getEmail());  
        namedParameterJdbcTemplate.update(INSERT_SQL, parameters, holder);  
        user.setId(holder.getKey().intValue());  
        return user;  
    }  
    public User findUserById(int id) {  
        Map parameters = new HashMap();  
        parameters.put("id", id);  
        return namedParameterJdbcTemplate.queryForObject(FETCH_SQL_BY_ID, parameters,  
            new UserMapper());  
    }  
}
```



Code JDBC ou JPA

On peut également se faire injecter les beans permettant de coder à un niveau plus bas :

- Au niveau JDBC, en se faisant injecter la DataSource
- Au niveau JPA, en se faisant injecter l'entityManager ou l'entityManagerFactory



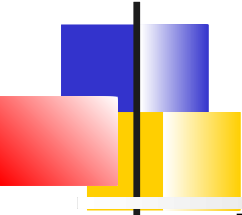
Service RestFul

Spring MVC et les APIs REST

Principes RESTFul

Dé/Sérialisation avec Jackson

Exceptions, CORS et OpenAPI



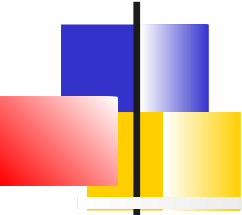
Introduction

Le module starter ***spring-boot-starter-web*** permet de charger le framework Spring MVC

Spring MVC permet de déclarer des beans de type

- ***@Controller*** ou ***@RestController***
- Dont les méthodes peuvent être associées à des requêtes HTTP via @RequestMapping

Dans la suite du support, nous nous concentrons sur les RestController



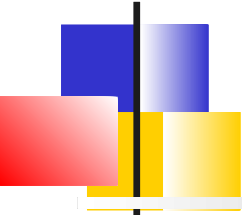
@RestController

L'annotation **@RestController** se positionne sur de simples classes dont les méthodes publiques sont généralement accessible via HTTP

@RestController

@RequestMapping("/api")

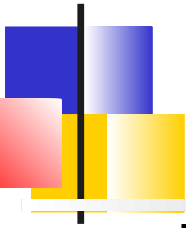
```
public class HelloWorldController {  
    @GetMapping("/helloWorld")  
    public String helloWorld() {  
        return "helloWorld";  
    }  
}
```



@RequestMapping

L'annotation **@RequestMapping**

- Au niveau de la classe, indique que toutes les méthodes du contrôleur seront relatives à ce chemin
 - Au niveau d'une méthode, l'annotation précise les attributs suivants :
 - **path** : Valeur fixe ou gabarit d'URI
 - **method** : Pour limiter la méthode à une action HTTP
 - **produce/consume** : Préciser le format des données d'entrée/sortie
- Dans le cadre d'une API Rest il n'est pas nécessaire de préciser ces attributs car le format est toujours JSON



Variantes @RequestMapping

Des variantes existent pour limiter à une seule méthode. Ce sont les annotation que l'on utilise en général dans une API Rest :

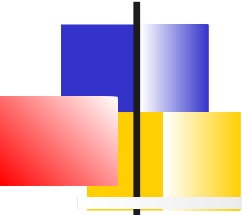
@GetMapping,

@PostMapping,

@PutMapping,

@DeleteMapping,

@PatchMapping

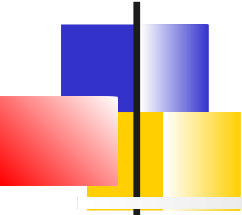


Types des arguments de méthode

Une méthode annoté via *@*Mapping* peut se faire injecter des arguments de type :

- La requête ou réponse HTTP (ServletRequest, HttpServletRequest, spring.WebRequest, ...)
- La session HTTP (HttpSession)
- La locale, la time zone
- La méthode HTTP
- L'utilisateur authentifié par HTTP (Principal)
- ..

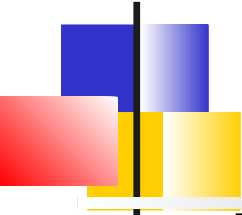
Si l'argument est d'un autre type, il nécessite des annotations afin que Spring puisse effectuer les conversions nécessaires à partir de la requête HTTP



Annotations sur les arguments de méthode

Ces annotations permettent d'associer un argument à une valeur de la requête HTTP. Les annotations principales utilisées dans le cadre d'une API Rest sont

- **@PathVariable** : Une partie de l'URI
- **@RequestParam** : Un paramètre HTTP (généralement passé par le caractère?)
- **@RequestBody** : Contenu de la requête au format Json qui sera converti en un objet Java
- **@RequestHeader** : Une entête HTTP
- **@RequestPart** : Une partie d'une requête multi-Part



Gabarits d'URI

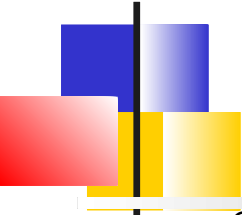
Un gabarit d'URI permet de définir des variables.

Ex :

http://www.example.com/users/{userId}

L'annotation **@PathVariable** associe la variable à un argument de méthode qui a le même nom

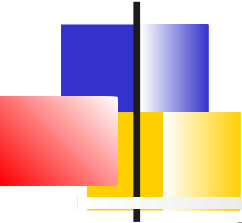
```
@GetMapping("/owners/{ownerId}")  
public String findOwner(@PathVariable  
    String ownerId) {
```



Paramètres HTTP avec *@RequestParam*

```
@RestController
@RequestMapping("/pets")
public class PetController {
    // ...
    @GetMapping
    public Pet getPet(@RequestParam("petId") int petId) {
        Pet pet = this.clinic.loadPet(petId);
        return pet;
    }
    // ...
}
```

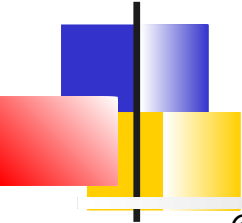
=> Exemple URL d'accès ***http://<server>/pets?petId=5***



@RequestBody et convertisseur

L'annotation ***@RequestBody*** permet de convertir le corps JSON de la requête dans un objet métier.

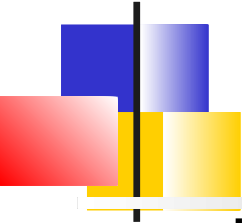
- Elle est typiquement utilisée sur des méthodes annotées *@PostMapping*, *@PutMapping*, *@PatchMapping*
- La conversion, appelée désérialisation, est effectuée par la librairie Jackson



Exemple *@RequestBody*

```
@RestController
@RequestMapping("/pets")
public class PetController {
    // ...

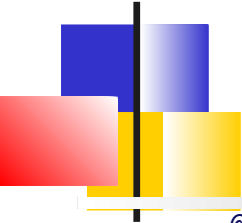
    @PostMapping
    public Pet savePet(@RequestBody Pet pet) {
        Pet pet = this.clinic.savePet(pet);
        return pet;
    }
    // ...
}
```



Types des valeurs de retours des méthodes

Les types des valeurs de retour possibles pour un contrôleur REST sont :

- Une **classe Modèle ou DTO** qui sera converti en JSON via la librairie Jackson.
Le code retour par défaut est alors 200
- Un objet ***ResponseEntity<T>*** permettant de positionner les codes retour et les entêtes HTTP voulues
- Il est également possible d'annoter la méthode avec ***@ResponseStatus*** pour indiquer le code de retour HTTP



Examples

@RestController

@RequestMapping(value="/users")

```
public class UsersController {
```

@GetMapping(value="/{id}")

```
public User getUser(@PathVariable Long id) { // ... }
```

@GetMapping(value="/{id}/customers")

```
List<Customer> getUserCustomers(@PathVariable Long id) { // ... }
```

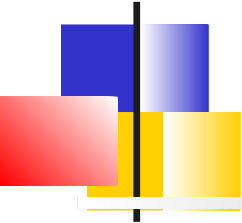
@DeleteMapping(value="/{id}")

```
public ResponseEntity<Void> deleteUser(@PathVariable Long id) {  
    return new ResponseEntity<>(HttpStatus.NO_CONTENT);  
}
```

@PostMapping

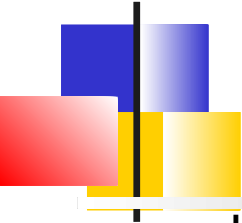
@ResponseStatus(HttpStatus.CREATED)

```
public User register(@Valid @RequestBody User user) {  
    return userRepository.save(user);  
}  
}
```



Service RestFul

Spring MVC et les APIs REST
Principes RESTFul
Dé/Sérialisation avec Jackson
Exceptions, CORS et OpenAPI



Auto-description

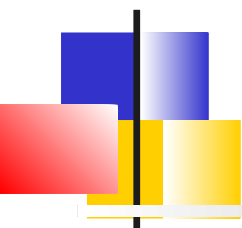
Une API bien conçue est compréhensible par un développeur sans qu'il ait à lire la documentation

- l'API est auto-descriptive.
- Une API se matérialise directement dans l'URL des requêtes HTTP envoyées au serveur exposant la ressource.

Exemple d'une requête HTTP sur une ressource de l'API de l'entreprise UBER :

GET https://api.uber.com/<version>/partners/125/payments

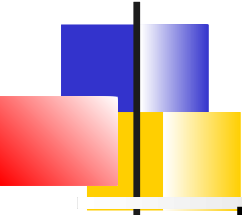
=> Un développeur sait intuitivement qu'il trouvera dans cette ressource les informations concernant les paiements reçues par le partenaire UBER (.).



Principe de Base

Une ressource $\leq \geq$ Une entité

URI	GET	PUT	POST	DELETE
Collections : <i>http://api.example.com/produits</i>	Liste tous les produits	Remplace la liste de produits avec une autre liste	Crée une nouvelle entrée dans la collection.	Supprime la collection.
Element, <i>http://api.example.com/produits/5</i>	Récupère le produit d'ID 5	Remplace ou crée l'élément	Traite l'élément comme une collection et y ajoute une entrée	Supprime l'élément



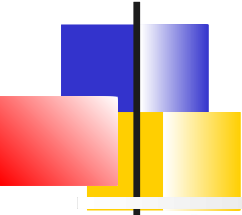
Règles de nommage

Pour les objets liés à l'entité principale. Il est recommandé d'utiliser une structure hiérarchique :
/objets/{objet_id}/sous_objets

GET /calendar/meetings/{meeting_id}/meeting_room

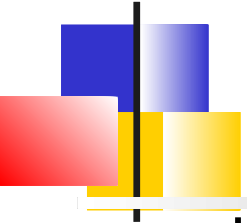
L'opération sur la ressource peut être précisée si la méthode HTTP ne suffit pas :

GET /calendar/meetings/{meeting_id}/attendees/search



Paramètres de requête

Paramètre	Utilisation	Exemple
Path Parameter	Pour l'identification seulement : - Uniquement un ID, toujours suivant l'entité à laquelle il se réfère - Paramètre obligatoire	{id} = 123 http://.../accounts/123/transactions
Query Parameter	Pour la gestion de résultat - filtrer, trier, ordonner, grouper les résultats (paramètres courts) : - Paramètres techniques optionnels - Valeurs sont définies et documentées dans la spécification de l'API	http://.../transactions ? from = NOW & sort = date:desc & limit = 50
Header	Pour la gestion du contexte d'application et de la sécurité - Utilisé par les navigateurs, les applications clientes et autres pour transmettre des informations sur le contexte de la demande - Utilisé pour transmettre les paramètres d'authentification <u>NB</u> : Ne pas utiliser pour transmettre les paramètres fonctionnels	Authorization : Bearer XXXXXXXX
Body	Pour les données fonctionnelles - Utilisé pour transmettre des informations fonctionnelles - Doit être un objet JSON	{ "name": "phone", "category": "tech", "max_price": 45 }

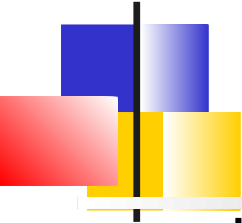


Codes retour

Les codes retours HTTP permettent de déterminer le résultat d'une requête ou d'indiquer une erreur au client.

Ils sont standards

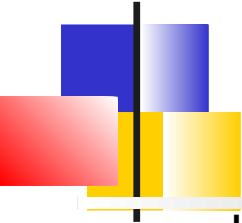
- **1xx** : Information
- **2xx** : Succès
- **3xx** : Redirection
- **4xx** : Erreur client
- **5xx** : Erreur serveur



Succès

Les codes retours « 2XX » sont les résultats des requêtes exécutées avec succès. Le code le plus courant est le code 200. Il en existe d'autres qui répondent à des cas plus précis.

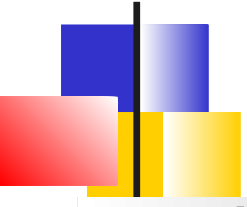
- **200 - OK** : Toute requête réussie
- **201 - Created & Location** : Création d'un nouvel objet. Le lien ou l'identifiant de la nouvelle ressource est envoyé dans la réponse
- **204 - No content** : Mettre à jour ou supprimer un objet (avec une réponse vide)
- **206 - Partial Content** : Une liste paginée d'objets par exemple



Erreurs client

Les codes retours « 4XX » indiquent que la requête envoyée par le client ne peut pas être exécutée par le serveur.

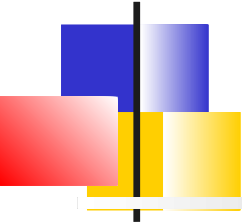
- **400 - Bad Request** : La requête est erronée. En général une mauvaise conversion
- **401 - Unauthorized** : La requête nécessite une authentification
- **403 - Forbidden** : Ressources non accessible pour l'utilisateur authentifié
- **404 - Not Found** : L'objet demandé n'existe pas
- **405 - Method Not Allowed** : L'URL est bonne mais la méthode HTTP
- **406 - Not Acceptable** : Les entêtes demandées ne peuvent pas être satisfaites. (Accept-Charset, Accept-Language)
- **409 - Conflict** : Par exemple : Tentative de création d'un nouveau utilisateur avec une adresse e-mail déjà existante
- **429 - Too Many Requests** : Le client a émis trop de requêtes dans un délai donné



Erreurs serveur

Les codes retours « 5XX » indiquent que le serveur a rencontré une erreur. Les types d'erreurs serveur les plus fréquents sont :

- **500 - Internal Server Error** : BUG !
- **501 - Not Implemented** : La méthode (GET, PUT, ...) n'est connue du serveur pour aucune ressource
- **502 - Bad Gateway ou Proxy Error** : La réponse du backend n'est pas comprise par l'API Gateway
- **503 - Service Unavailable** : API hors service, en maintenance, ...
- **504 - Gateway Time-out** : Timeout dépassé

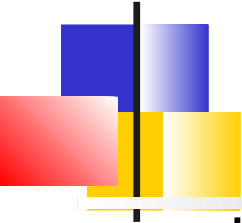


Service RestFul

Spring MVC et les APIs REST
Principes RESTFul

Dé/Sérialisation avec Jackson

Exceptions, CORS et OpenAPI



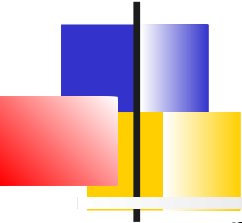
Sérialisation JSON

Un des principales problématiques des back-end Spring et la conversion des objets du domaine au format JSON.

Des librairies spécialisés sont utilisées (Jackson, Gson), elles permettent de bénéficier de comportement par défaut.

Mais, généralement le développeur doit régler certaines problématiques :

- Boucle infinie pour les relations bidirectionnelles entre classes du modèle
- Adaptation aux besoins de l'interface du front-end
- Optimisation du volume de données échangées
- Format des dates

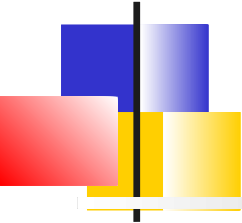


Comportement par défaut

```
public class Member {  
    private long id;  
    private String nom,prenom;  
    private int age;  
    private Date registeredDate;  
}
```

Devient :

```
{  
    "id": 5,  
    "nom": "Dupont",  
    "prenom": "Gaston",  
    "age": 71,  
    "registeredDate": 1645271583944  
}
```

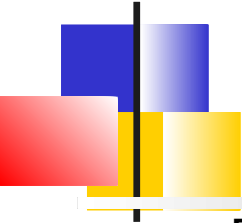


Concepts Jackson

Avec Jackson, les sérialisations/désérialisations sont effectuées généralement par des ***ObjectMapper***

```
// Sérialisation
Member m = memberRepository.findById(4l) ;
ObjectMapper objectMapper = new ObjectMapper() ;
String jsonString = objectMapper.writeValueAsString(m) ;
...
// Désérialisation
String jsonString= "{\n\"id\" : 5,\n" + ... + "}" ;
Member m2 = objectMapper.readValue(jsonString) ;
```

Dans un contexte SpringBoot, on utilise rarement l'objet *ObjectMapper* directement ... mais on influence son comportement par des annotations.



Solutions aux problématiques de sérialisation

Pour adapter la sérialisation par défaut de Jackson à ses besoins, 4 alternatives :

- Créer des **classes DTO spécifiques**.

La couche Service transforme les classes Entité provenant de la couche Repository en des classes Data Transfer Object spécialisées pour la sérialisation

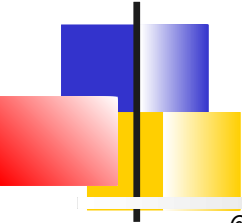
- Utiliser les **annotations** proposées par Jackson

Sur les classes DTO ou les classes Entité, utiliser les annotations Jackson pour s'adapter au besoin de la sérialisation

- Utiliser l'annotation **@JsonView**

Le même objet Entité ou Dto peut alors être sérialisé différemment en fonction des cas d'usage

- Implémenter ses propres Sérialiseur/Désérialiseur. Spring propose l'annotation **@JsonComponent**



Exemple DTO

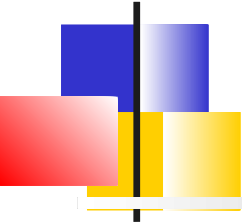
```
@Service
public class UserService {
    @Autowired UserRepository userRepository;
    @Autowired RolesRepository rolesRepository;

    UserDto retrieveUser(String login) {
        User u = userRepository.findByLogin(login);
        List<Role> roles = rolesRepository.findByUser(u);
        return new UserDto(u, roles);
    }
}

-----

public class UserDto {
    private String login, email, nom, prenom;
    List<Role> roles;

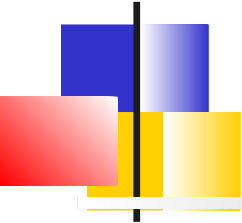
    public UserDto(User user, List<Role> roles) {
        login = user.getLogin(); email = user.getEmail();
        nom = user.getNom(); prenom = user.getPrenom();
        this.roles = roles;
    }
}
```



Format de Dates

Pour avoir une représentation String des dates selon les bon vouloir du front-ent, la solution est plus souple est d'utiliser **@JsonFormat**

```
public class Event {  
    public String name;  
    @JsonFormat(shape = JsonFormat.Shape.STRING,  
        pattern = "dd-MM-yyyy hh:mm:ss")  
    public Date eventDate;  
}
```

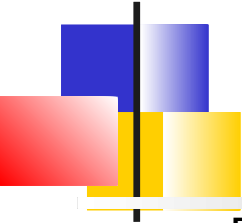


Relations bidirectionnelles

Le problème

```
public class User {  
    public int id;  
    public String name;  
    public List<Item> userItems;  
}  
  
public class Item {  
    public int id;  
    public String itemName;  
    public User owner;  
}
```

Lorsque Jackson sérialise l'une des 2 classes, il tombe dans une boucle infinie



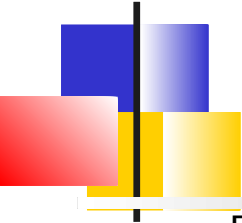
Relations bidirectionnelles

Une solution

En annotant les 2 classes avec **@JsonManagedReference** et **@JsonBackReference**

```
public class User {  
    public int id;  
    public String name;  
    @JsonManagedReference  
    public List<Item> userItems;  
}  
  
public class Item {  
    public int id;  
    public String itemName;  
    @JsonBackReference  
    public User owner;  
}
```

La propriété userItems est sérialisé mais pas owner



Relations bidirectionnelles

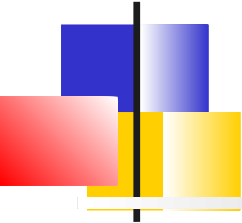
Une autre solution

En annotant les classes avec `@JsonIdentityInfo` qui demande à Jackson de sérialiser une classe juste avec son ID

```
@JsonIdentityInfo(  
    generator = ObjectIdGenerators.PropertyGenerator.class, property = "id")  
public class User {...}  
  
@JsonIdentityInfo(  
    generator = ObjectIdGenerators.PropertyGenerator.class, property = "id")  
public class Item { ... }
```

Sérialisation d'un Item :

```
{  
  "id":2,  
  "itemName":"book",  
  "owner":  
  {  
    "id":1,  
    "name":"John",  
    "userItems":[2]  
  }  
}
```

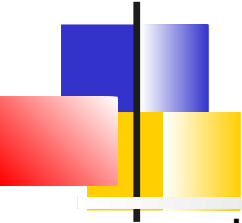


Relations bidirectionnelles

Une autre solution

En annotant les classes avec **@JsonIgnore** on demande à Jackson de ne pas sérialiser une propriété

```
public class User {  
    public int id;  
    public String name;  
    public List<Item> userItems;  
}  
  
public class Item {  
    public int id;  
    public String itemName;  
    @JsonIgnore  
    public User owner;  
}
```



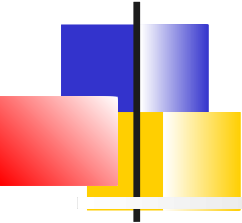
@JsonView

Les différentes représentations d'un dto sont regroupées dans une classe. Des relations d'héritages peuvent être définies entre les différentes vues.

```
public class CompanyViews {  
    public static class Normal{};  
    public static class Manager extends Normal{};  
    public static class HR extends Normal{};  
}
```

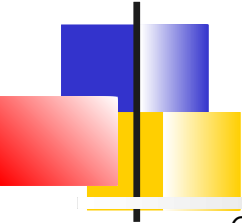
Les classes sont ensuite référencées via l'annotation **@JsonView** :

- Sur les classes du modèles :
 - Quel attribut est sérialisé lorsque telle vue est activée ?
- Sur les méthodes des contrôleurs :
 - Quelle vue doit être utilisée lors de la sérialisation de la valeur de retour de cette méthode ?



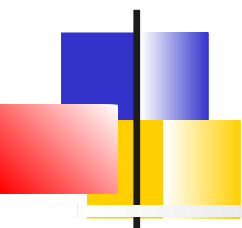
@JsonView : Annotations sur le modèle

```
public class Staff {  
    @JsonView(CompanyViews.Normal.class)  
    private String name;  
    @JsonView(CompanyViews.Normal.class)  
    private int age;  
    // 2 vues  
    @JsonView({CompanyViews.HR.class, CompanyViews.Manager.class})  
    private String[] position;  
    @JsonView(CompanyViews.Manager.class)  
    private List<String> skills;  
    @JsonView(CompanyViews.HR.class)  
    private Map<String, BigDecimal> salary;
```



Activation d'une vue

```
@RestController
public class StaffController {
    @GetMapping
    @JsonView(CompanyViews.Normal.class)
    public List<Staff> findAll() {
    }
    ...
    ObjectMapper mapper = new ObjectMapper();
    Staff staff = createStaff();
    try {
        String normalView =
            mapper.writerWithView(CompanyViews.Normal.class).writeValueAsString(staff);
```



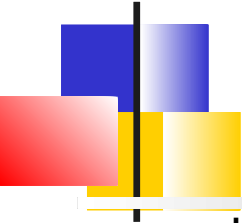
Autres annotations Jackson

`@JsonProperty`, `@JsonGetter`, `@JsonSetter`,
`@JsonAnyGetter`, `@JsonAnySetter`, `@JsonIgnore`,
`@JsonIgnoreProperty`, `@JsonIgnoreType` :
Permettant de définir les propriétés JSON

`@JsonRootName` : Arbre JSON

`@JsonSerialize`, `@JsonDeserialize` : Indique des
dé/sérialiseurs spécialisés

... .

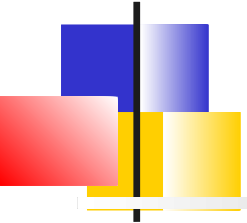


Sérialiseur spécifique

L'annotation Spring **@JsonComponent** facilite l'enregistrement de sérialiseurs/désérialiseurs Jackson

Elle doit être placée sur des implémentations de *JsonSerializer* et *JsonDeserializer* ou sur des classes contenant des inner-class de ce type

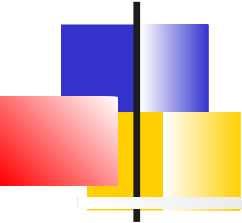
```
@JsonComponent
public class Example {
    public static class Serializer extends JsonSerializer<SomeObject>
    {
        // ...
    }
    public static class Deserializer extends
        JsonDeserializer<SomeObject>
    {
        // ...
    }
}
```



OpenInView

Lors d'une application Web, Spring Boot enregistre par défaut l'intercepteur ***OpenEntityManagerInViewInterceptor*** afin d'appliquer le pattern "Open EntityManager in View" permettant d'éviter les LazyException dans les vues

Si ce n'est pas le comportement voulu :
spring.jpa.open-in-view = false

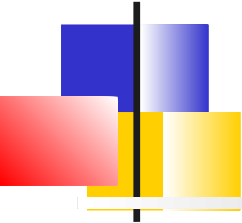


Service RestFul

Spring MVC et les APIs REST
Principes RESTFul

Dé/Sérialisation avec Jackson

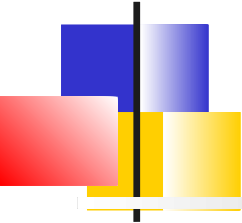
Exceptions, CORS et OpenAPI



Personnalisation de la configuration Spring MVC

La personnalisation de la configuration par défaut de SpringBoot peut être effectuée en définissant un bean de type ***WebMvcConfigurer*** et en surchargeant les méthodes proposées.

Dans le cadre d'une API Rest, une méthode permet de configurer le CORS

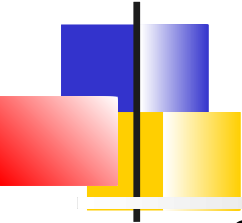


Exemple Cross-origin

Le CORS peut se configurer globalement en surchargeant la méthode *addCorsMapping* de *WebMvcConfigurer* :

```
@Configuration
public class MyConfiguration implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**").allowedOrigins("*");
    }
}
```

A noter qu'il est également possible de configurer le cors individuellement sur les contrôleurs via l'annotation **@CrossOrigin**



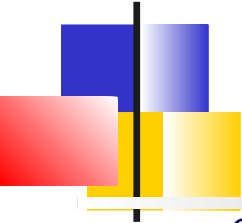
Gestion des erreurs

Spring Boot associe /error à la page d'erreur globale de l'application

- Un comportement par défaut en REST ou en Web permet de visualiser la cause de l'erreur

Pour remplacer le comportement par défaut pour un service Restful

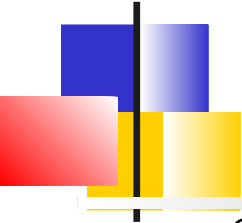
- L'annotation ***ResponseStatus*** sur une exception métier lancée par un contrôleur
- Utiliser la classe ***ResponseStatusException*** pour associer un code retour à une Exception
- Ajouter une classe annotée par ***@RestControllerAdvice*** pour centraliser la génération de réponse lors d'exception



Example

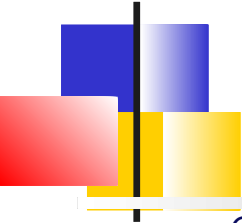
@ResponseStatus(value = HttpStatus.NOT_FOUND)

```
public class MyResourceNotFoundException extends RuntimeException {  
    public MyResourceNotFoundException() {  
        super();  
    }  
    public MyResourceNotFoundException(String message, Throwable cause) {  
        super(message, cause);  
    }  
    public MyResourceNotFoundException(String message) {  
        super(message);  
    }  
    public MyResourceNotFoundException(Throwable cause) {  
        super(cause);  
    }  
}
```



ResponseStatusException

```
@GetMapping(value =("/{id}")
public Foo findById(@PathVariable("id") Long id,
    HttpServletResponse response)
{
    try {
        Foo resourceById =
            RestPreconditions.checkFound(service.findOne(id));
        eventPublisher.publishEvent(new
            SingleResourceRetrievedEvent(this, response));
        return resourceById;
    } catch (MyResourceNotFoundException exc) {
        throw new ResponseStatusException(
            HttpStatus.NOT_FOUND, "Foo Not Found", exc);
    }
}
```



Exemple

@RestControllerAdvice

@RestControllerAdvice

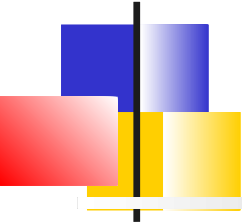
```
public class NotFoundAdvice extends ResponseEntityExceptionHandler {
```

```
@ExceptionHandler(value = {MemberNotFound.class,  
    DocumentNotFound.class})
```

```
    ResponseEntity<Object> handleNotFoundException(HttpServletRequest  
        request,  
        Throwable ex) {  
        return new ResponseEntity<Object>(  
            "Entity was not found", new HttpHeaders(), HttpStatus.NOT_FOUND);  
        }  
    }
```

@Override

```
    protected ResponseEntity<Object>  
        handleMethodArgumentNotValid(MethodArgumentNotValidException ex,  
            HttpHeaders headers, HttpStatus status, WebRequest request) {  
        return new ResponseEntity<Object>(  
            ex.getMessage(), new HttpHeaders(), HttpStatus.BAD_REQUEST);  
        }  
    }
```



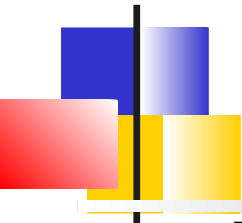
SpringDoc et OpenAPI

SpringDoc est un outil qui simplifie la génération et la maintenance de la documentation des API REST

Il est basé sur la spécification OpenAPI 3 et s'intègre avec Swagger-UI

Il suffit de placer la dépendance dans le fichier de build :

```
org.springdoc :springdoc-openapi-starter-webmvc-ui :${version}
```



Fonctionnalités

Par défaut,

- La description OpenAPI est disponible à :
<http://localhost:8080/v3/api-docs/>
- L'interface Swagger à :
<http://localhost:8080/swagger-ui.html>

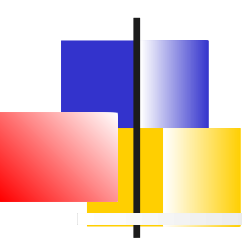
SpringDoc prend en compte

- les annotations javax.validation positionnées sur les DTOs
- Les Exceptions gérées par les @ControllerAdvice
- Les annotations de OpenAPI

<https://javadoc.io/doc/io.swagger.core.v3/swagger-annotations/latest/index.html>

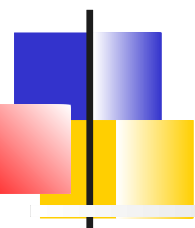
SpringDoc peut être désactivé via la propriété :

springdoc.api-docs.enabled=false



Interactions entre services

RestClient Messaging

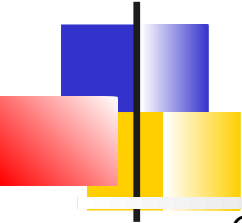


Supports pour les appels Rest

Spring Boot 3.x propose plusieurs approches pour effectuer des appels HTTP :

- **WebClient** : API réactive utilisable également en mode synchrone
- **RestClient** : nouveau client HTTP introduit en Spring Framework 6.1.
- **RestTemplate** : historique, toujours supporté mais en évolution minimale.

Aucun de ces clients n'est auto-exposé directement en bean final : l'application doit créer l'instance souhaitée via le builder fourni.



Example Construction

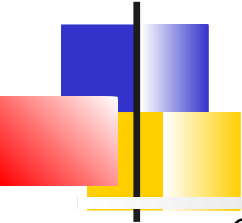
@Configuration

```
public class RestClientConfig {
```

```
@Bean public RestClient restClient(  
    RestClient.Builder builder) {
```

```
    return builder.baseUrl("https://api.example.com")  
        .defaultHeader("Authorization", "Bearer YOUR_TOKEN")  
        .build();
```

```
} }
```

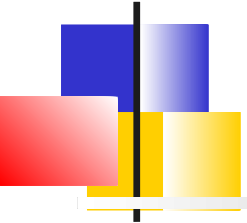


Exemple Usage

```
@Service
@RequiredArgsConstructor
public class RestService {

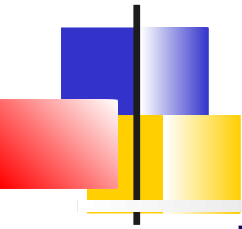
    private final RestClient client;

    public User loadUser(long id) {
        return client
            .get()
            .uri("/api/users/{id}", id)
            .retrieve()
            .body(User.class);
    }
}
```



Exemple Usage (2)

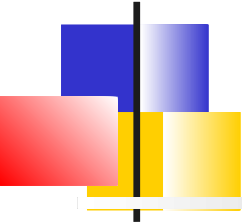
```
public User loadUser(long id) {  
    return client.get()  
        .uri("/api/users/{id}", id)  
        .retrieve()  
        .onStatus(HttpStatusCode::is4xxClientError, (req, res) ->  
            {  
                throw new IllegalArgumentException("User not found");  
            })  
        .onStatus(HttpStatusCode::is5xxServerError, (req, res) ->  
            {  
                throw new IllegalStateException("Remote error");  
            })  
        .body(User.class);  
}
```



WebClient

WebClient est l'interface apportée par Spring Webflux permettant d'effectuer les requêtes REST réactive

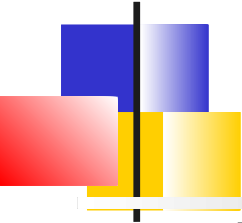
La solution offre du support pour les interactions synchrones et asynchrones,



Builder

```
// Builder
```

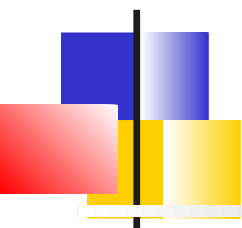
```
WebClient client = WebClient.builder()  
    .baseUrl("http://localhost:8080")  
    .defaultCookie("cookieKey", "cookieValue")  
    .defaultHeader(HttpHeaders.CONTENT_TYPE,  
                    MediaType.APPLICATION_JSON_VALUE)  
    .defaultUriVariables(Collections.singletonMap("url",  
                                                  "http://localhost:8080"))  
    .build();
```



Préparation de la requête

La préparation de la requête consiste à préciser la méthode HTTP, l'URL, le corps et les entêtes.

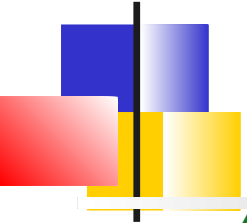
```
requestSpec = client.post()  
    .uri("/resource")  
    .bodyValue("data")  
    .header(HttpHeaders.CONTENT_TYPE,  
        MediaType.APPLICATION_JSON_VALUE)  
    .accept(MediaType.APPLICATION_JSON,  
        MediaType.APPLICATION_XML)
```



Récupération de la réponse

Pour récupérer la réponse on peut utiliser ***exchangeToMono*** et ***exchangeToFlux*** qui permettent d'inspecter la réponse (entête, code status,)

Ou tout simplement ***retrieve*** qui permet de récupérer le corps de la réponse sous forme de classe réactive.



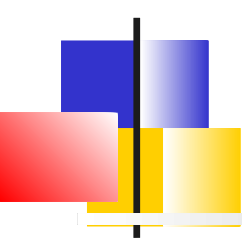
Examples

// exchangeToMono

```
Mono<String> response = requestSpec.exchangeToMono(response -> {  
    if (response.statusCode().equals(HttpStatus.OK)) {  
        return response.bodyToMono(String.class);  
    } else if (response.statusCode().is4xxClientError()) {  
        return Mono.just("Error response");  
    } else {  
        return response.createException()  
            .flatMap(Mono::error);  
    }  
});
```

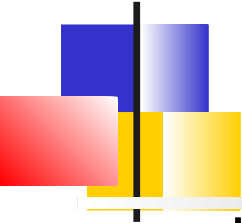
// Retrieve simple

```
Mono<String> response = headersSpec.retrieve()  
    .bodyToMono(String.class);
```

Interactions entre services

RestClient
Messaging



Introduction

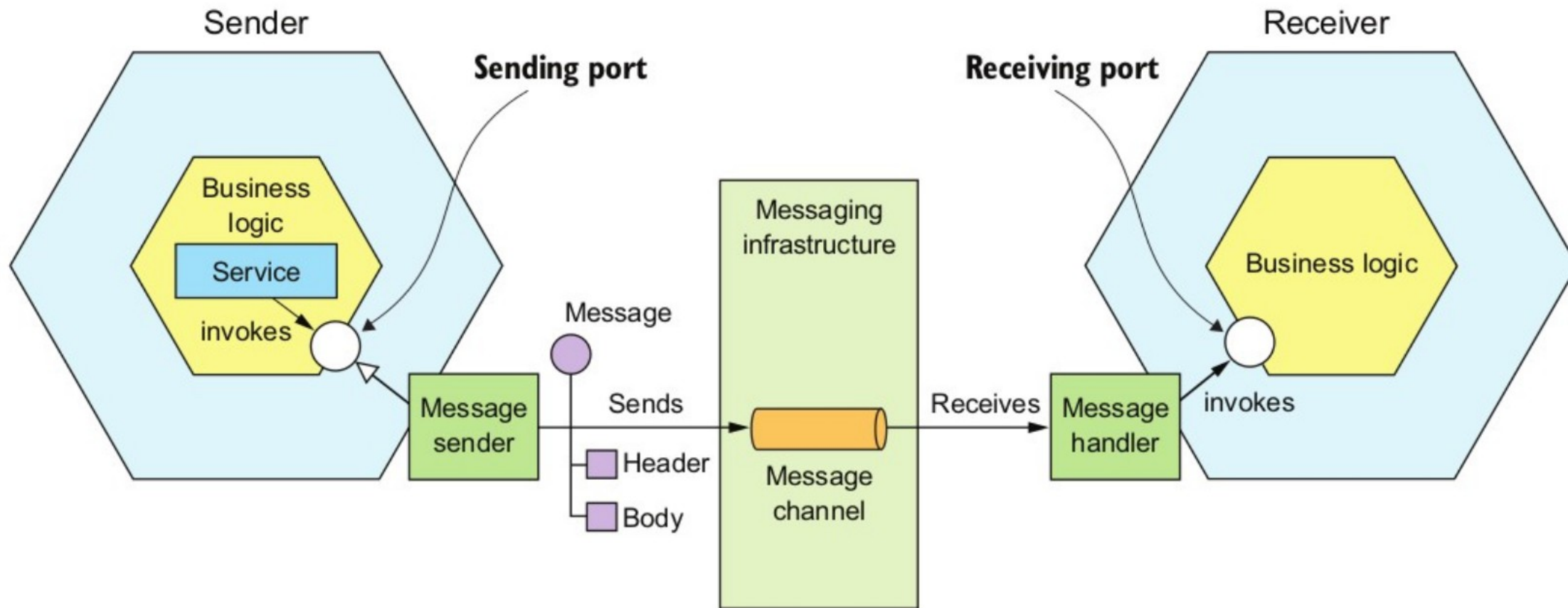
Les communications asynchrones entre processus apportent plusieurs avantages :

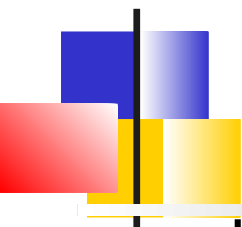
- Découplage du producteur et consommateur de message
- Scaling et montée en charge
- Implémentation de patterns de micro-services Saga, Event-sourcing Pattern

Des difficultés :

- Gestion de l'asynchronisme
- Mise en place et exploitation d'un message broker

Architecture



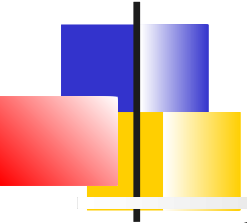


Sémantique des messages

Un message est constitué d'entêtes (ensemble de clés-valeurs), d'un corps de message et éventuellement une clé (Kafka)

On distingue 3 types de messages :

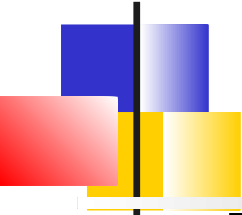
- **Document** : Un message générique ne contenant que des données. Le récepteur décide comment l'interpréter
- **Commande** : Un message spécifiant l'action à invoquer et ses paramètres d'entrée
- **Événement** : Un message indiquant que quelque chose vient de se passer. Typiquement un événement métier



Canaux de messages

2 types de canaux :

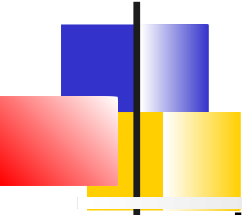
- ***Point-to-point*** : Le canal délivre le message à un des consommateurs lisant le canal.
Ex : Envoie d'un message commande
- ***PubAndSub*** : Le canal délivre le message à tous les consommateurs attachés (les abonnés)



Styles d'interaction

Tous les styles d'interactions sont supportés :

- Requête/Réponse synchrone.
Le client attend la réponse
- Requête/Réponse asynchrone
Le client est notifié lorsque la réponse arrive
- One way notification
Le client n'attend pas de réponse
- Publish and Subscribe :
Le producteur n'attend pas de réponse
- Publish et réponse asynchrones
Le producteur est notifié lorsque les réponses arrivent



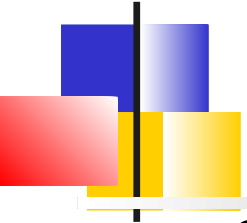
Messages broker

Un message broker est un intermédiaire par lequel tous les messages transitent

- L'émetteur n'a pas besoin de connaître l'emplacement réseau du récepteur
- Le message broker bufferise les messages

Implémentations courantes :

- ActiveMQ
- RabbitMQ
- Kafka
- AWS Kinesis



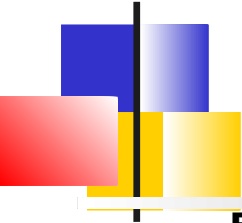
Offre Spring

Starter messaging pur :

- RabbitMQ, ActiveMQ, Kafka, ActiveMQ, Artemis, Solace PubSub

Architecture micro-services event-driven

- Kafka Stream
- Spring Cloud Stream
- Spring Data Flow



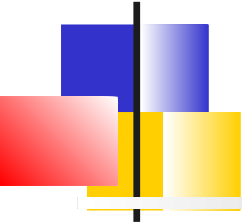
Exemple spring-kafka

Envoi de message

```
@Value("${app.my-channel}")
String ORDER_STATUS_CHANNEL;
@Autowired
KafkaTemplate<Long, DomainEvent> kafkaOrderTemplate;
public Order doService(Domain model) {
...
DomainEvent event = new DomainEvent(model);
kafkaOrderTemplate.send(ORDER_STATUS_CHANNEL, event);
...
}
```

Réception de message :

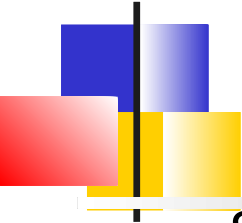
```
@KafkaListener(topics = "#{ '${app.my-channel}' }", id = "oneHandler")
public void handleEvent(DomainEvent domainEvent) {
...
}
```



Spring Security

Principes

Modèles stateful/stateless
Auto-configuration Spring Boot
OpenIdConnect et OAuth2



Spring Security

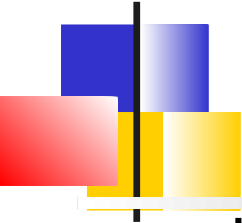
Spring Security gère principalement 2 domaines de la sécurité :

L'authentification : S'assurer de l'identité de l'utilisateur ou du système

L'autorisation : Vérifier que l'utilisateur ou le système ait accès à une ressource.

Il facilite la mise en place de la sécurité en :

- S'intégrant à des fournisseurs d'authentification standard (LDAP, OpenIDConnect, Kerberos, PAM, CAS) OU Complètement spécialisés
- En permettant la configuration des contraintes d'accès aux URLs et aux méthodes des services métier



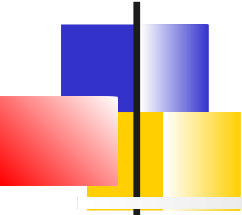
Spring Security

La configuration par défaut de la sécurité web peut être provoquée par l'annotation **@EnableWebSecurity** ou par **SpringBoot**

Le bean **springSecurityFilterChain** encapsule une chaîne de filtres interceptant toutes les requêtes HTTP.

- Chaque filtre est responsable d'un aspect de la sécurité.
- La chaîne de filtre est hautement configurable et s'adapte à toutes les approches

Si en plus on désire ajouter de la sécurité au niveau des méthodes, il faut explicitement l'activer avec **@EnableGlobalMethodSecurity**



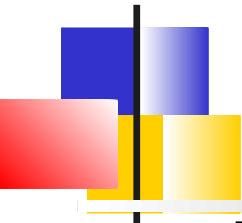
Quelques Filtres communs de *springSecurityFilterChain*

UsernamePasswordAuthenticationFilter : Répond par défaut à /login, récupère les paramètres username et password et appelle le gestionnaire d'authentification

SessionManagementFilter : Gestion de la collaboration entre la session http et la sécurité

BasicAuthenticationFilter : Traite les entêtes d'autorisation d'une authentification basique

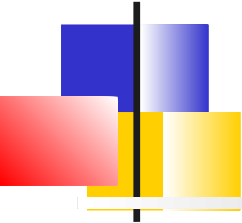
SecurityContextPersistenceFilter : Responsable de stocker le contexte de sécurité (par exemple dans la session http)



Personnalisation de la sécurité

La personnalisation de la configuration consiste :

- A personnaliser le filtre ***springSecurityFilterChain*** en créant un bean de type SecurityFilterChain la classe ***HttpSecurity*** est un builder facilitant sa création
- À personnaliser l'authentification :
 - en créant un bean de type ***AuthenticationManager***
La classe AuthenticationManagerBuilder facilite la création de Realm (inMemory, jdbc, ldap, ...)
 - ou complètement personnalisé par l'implémentation d'un bean ***UserDetailsService***



Exemple

```
// Ex : Modèle stateful
```

```
@Bean
```

```
public SecurityFilterChain filterChain(HttpSecurity http)
                                throws Exception {
```

```
    http
```

```
        .authorizeRequests(authorize ->
```

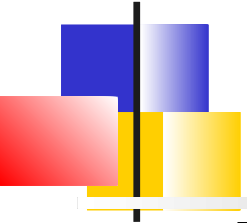
```
            authorize.anyRequest().authenticated())
```

```
                .formLogin(withDefaults())
```

```
                .httpBasic(withDefaults());
```

```
    return http.build();
```

```
}
```



Debug de la sécurité

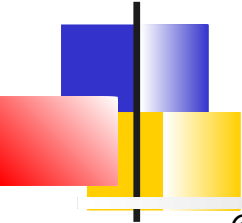
Pour debugger la configuration :

- La configuration de *springSecurityFilterChain* est affichée sur la console au démarrage de Spring

Pour debugger l'exécution :

- Activer les traces de DEBUG :

```
logging.level.org.springframework.security=DEBUG
```

Exemple : Configuration du gestionnaire d'authentification

@Configuration

```
public class SecurityConfiguration {
```

```
// Implémentation UserDetailsService
```

```
@Bean
```

```
public InMemoryUserDetailsManager userDetailsService() {
```

```
    UserDetails user = User.withDefaultPasswordEncoder()
```

```
        .username("user")
```

```
        .password("password")
```

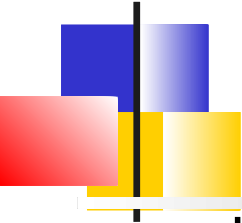
```
        .roles("USER")
```

```
        .build();
```

```
    return new InMemoryUserDetailsManager(user);
```

```
}
```

```
}
```



Personnalisation de UserDetailsService

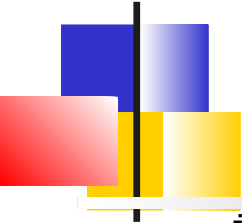
L'implémentation de ***UserDetailsService*** peut être complètement personnalisée

L'interface contient une seule méthode :

```
public UserDetails loadUserByUsername(String login)
    throws UsernameNotFoundException
```

Elle est responsable de retourner, à partir d'un login, un objet de type ***UserDetails*** encapsulant le mot de passe et les rôles.

C'est le framework qui effectue la vérification du mot de passe.

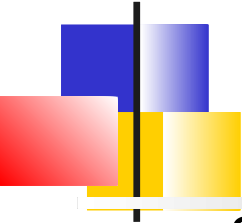


Exemple

```
import org.springframework.security.core.userdetails.User ;
...
@Service
public class UserDetailsServiceImpl implements UserDetailsService{
    @Autowired
    private AccountRepository accountRepository;

    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String login) throws
        UsernameNotFoundException {
        Account account = accountRepository.findByLogin(login).orElseThrow(
            () → new UsernameNotFoundException("Invalides login/mot de passe"));

        Set<GrantedAuthority> grantedAuthorities = new HashSet<>();
        for (Role role : account.getRoles()){
            grantedAuthorities.add(new SimpleGrantedAuthority(role.getLibelle()));
        }
        return new User(account.getLogin(), account.getPassword(),
            grantedAuthorities);
    }
}
```



Password Encoder

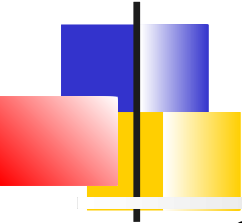
Spring Security 5 nécessite que les mots de passes soient encodés

Il faut alors définir un bean de type ***PasswordEncoder***

L'implémentation recommandée est *BcryptPasswordEncoder*

@Bean

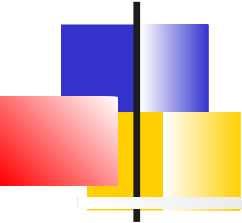
```
PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```



{noop}

Si les mots de passes sont stockés en clair, il faut les préfixer par ***{noop}*** afin que Spring Security n'utilise pas d'encodeur

```
public UserDetails loadUserByUsername(String login) throws
    UsernameNotFoundException {
    Member member = memberRepository.findByEmail(login);
    if ( member == null )
        throw new UsernameNotFoundException("Invalides login/mot de
        passe");
    Set<GrantedAuthority> grantedAuthorities = new HashSet<>();
    return new User(member.getEmail(), "{noop}" + member.getPassword(),
        grantedAuthorities);
}
```



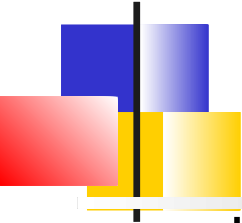
Spring Security

Principes

Modèles stateful/stateless

Auto-configuration Spring Boot

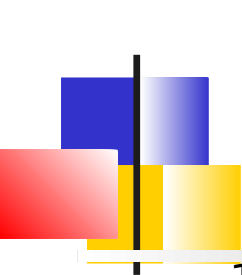
OpenIdConnect et OAuth2



Modèles

Une fois authentifié, il y a 2 stratégies de gestion de l'authentification :

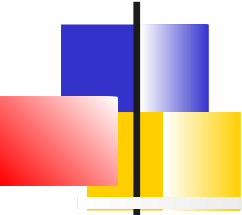
- L'approche **stateful** ou l'authentification est stockée dans la sessions http de l'utilisateur (cookie en général).
C'est la technique des application web legacy ou d'application nécessitant pas de scalabilité
- L'approche **stateless**. L'authentification n'est pas stocké côté serveur et le client doit fournir la preuve de son authentification à chaque fois sous la forme d'un jeton.
Le service est alors facilement scalable. Utilisé majoritairement dans les services Restful



Authentication stateful

Appli web standard ou monolithique Rest

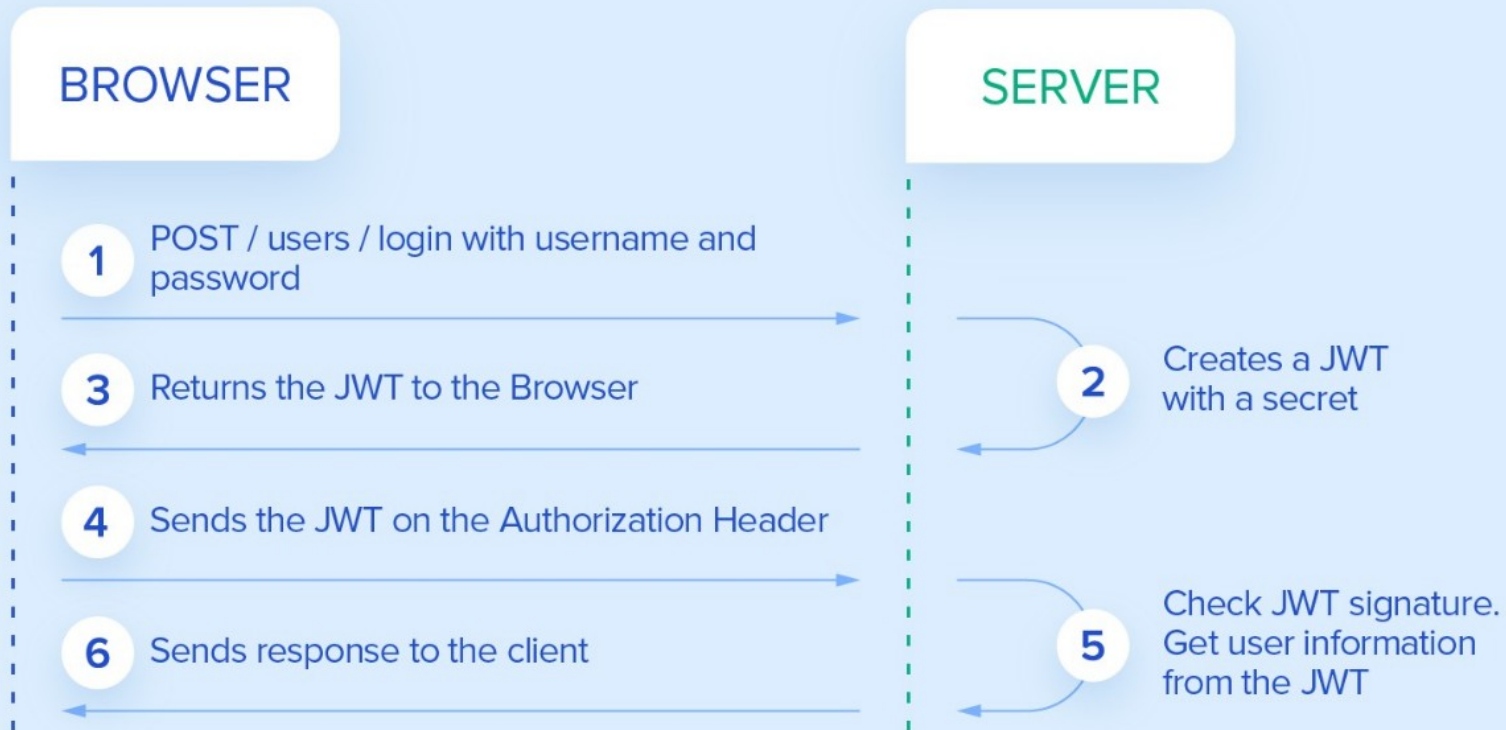
1. Le client demande une ressource protégée.
2. Le serveur renvoie une réponse indiquant que l'on doit s'authentifier :
 - 1) En redirigeant vers une page de login
 - 2) En fournissant les entêtes pour une authentication basique du navigateur .
3. Le navigateur renvoie une réponse au serveur :
 - 1) Soit le POST de la page de login
 - 2) Soit les entêtes HTTP d'authentification.
4. Le serveur décide si les créidentiels sont valides :
 - 1) Si oui. L'authentification est stockée dans la session, la requête originelle est réessayée, si les droits sont suffisants la page est retournée sinon un code 403
 - 2) Sinon, le serveur redemande une authentication.
5. L'objet Authentication contenant l'utilisateur et ses rôles est présent dans la session.

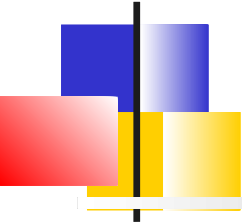


Authentication stateless API REST - microservice

1. Le client demande une ressource protégée.
2. Le serveur renvoie une réponse indiquant que l'on doit s'authentifier en envoyant une réponse 401.
3. Le navigateur propose un formulaire de login puis envoie le formulaire sur un serveur d'authentification (peut être différent que le serveur d'API)
4. Le serveur d'authentification décide si les créden-tiels sont valides :
 - 1) Si oui. Il génère un token avec un délai de validité
 - 2) Si non, le serveur redemande une authentification .
5. Le client récupère le jeton et l'associe à toutes les requêtes vers l'API
6. Le serveur de ressources décrypte le jeton et déduit les droits de l'utilisateur. Il autorise ou interdit l'accès à la ressource

Example





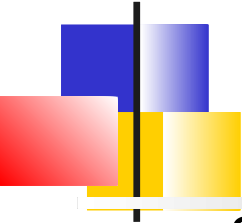
Spring Security

Principes

Modèles stateful/stateless

Auto-configuration Spring Boot

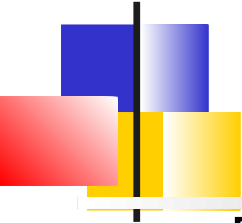
OpenIdConnect et OAuth2



Apports de SpringBoot

Si Spring Security est dans le classpath, la configuration par défaut :

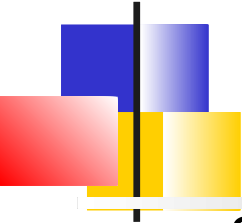
- Sécurise toutes les URLs de l'application web par l'authentification formulaire
- Un gestionnaire d'authentification simpliste est configuré pour permettre l'identification d'un unique utilisateur.
Le login par défaut est **user**, le mot de passe aléatoire est généré et affiché sur la console
Les propriétés par défaut peuvent être changées :
`security.user.name= myUser`
`security.user.password=secret`



Autres fonctionnalités par défaut

D'autres fonctionnalités sont automatiquement obtenues :

- Les chemins pour les ressources statiques standard sont ignorées (/css/**, /js/**, /images/**, /webjars/** et **/favicon.ico).
- Les événements liés à la sécurité sont publiés vers ApplicationEventPublisher via DefaultAuthenticationEventPublisher
- Des fonctionnalités communes de bas niveau (HSTS, XSS, CSRF, caching)



Autres fonctionnalités par défaut

SSL peut être configuré via les propriétés préfixées par ***server.ssl.****

Par exemple :

```
server.port=8443
```

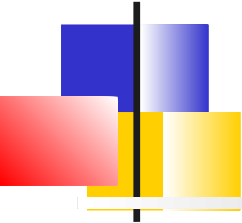
```
server.ssl.key-store=classpath:keystore.jks
```

```
server.ssl.key-store-password=secret
```

```
server.ssl.key-password=another-secret
```

Par défaut si SSL est configuré, le port 8080 disparaît.

Si l'on désire les 2, il faut configurer explicitement le connecteur réseau



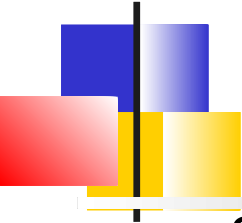
Spring Security

Principes

Modèles stateful/stateless

Auto-configuration Spring Boot

OpenIdConnect et OAuth2



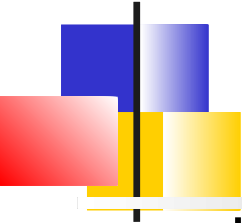
Starters OpenID

Spring fournit plusieurs starters relatifs à OpenID :

OAuth2 Client : Intégration pour obtenir un jeton auprès d'un serveur d'authentification (Google, Github, Facebook, Keycloak, ...)

OAuth2 Resource server : Application permettant de définir des ACLs par rapport aux scopes client et aux rôles contenu dans des jetons OAuth

OAuth2 Authorization server : Application permettant de délivrer des jetons

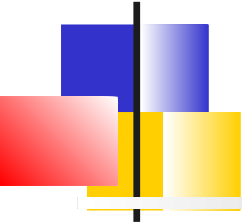


Login OAuth

Le starter oauht2-client facilite la configuration des fournisseurs classiques : Google, Github, Facebook, ... ainsi que les solutions d'entreprise Keycloak, OAuth2 Authorization Server

@Bean

```
public SecurityWebFilterChain  
    securityWebFilterChain(ServerHttpSecurity http) {  
  
    return http.authorizeExchange() .anyExchange().authenticated()  
        .and().oauth2Login()  
        .and().csrf().disable() .build();  
  
}
```



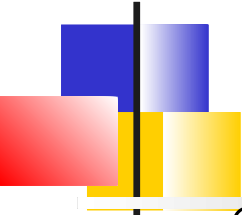
Configuration

Configuration Google :

```
spring:
  security:
    oauth2:
      client:
        registration: google:
        client-id: google-client-id
        client-secret: google-client-secret
```

Configuration Keycloak

```
spring :
  security:
    oauth2:
      client:
        provider: keycloak:
        issuer-uri: http://keycloak/realms//
        registration:
          spring-app:
            provider: keycloak
            client-id: spring-app
            client-secret: 57abb4f6-5130-4c73-9545-6d377dd947cf
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/keycloak"
            scope :openid
```

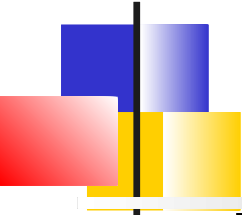


Accès à l'utilisateur loggé.

```
@GetMapping("/oidc-principal")
public OidcUser getOidcUserPrincipal(
    @AuthenticationPrincipal OidcUser principal) {
    return principal;
}

...
Authentication authentication =
    SecurityContextHolder.getContext().getAuthentication();
if (authentication.getPrincipal() instanceof OidcUser)
{
    OidcUser principal =
        ((OidcUser)authentication.getPrincipal());

    // ...
}
```



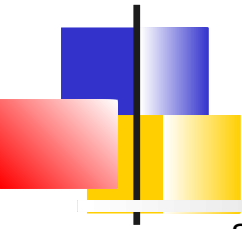
Serveur de ressource

Dépendance :

spring-boot-starter-oauth2-resource-server

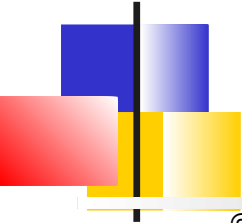
Le serveur de ressources doit vérifier la signature du jeton pour s'assurer qu'il a été issu par le bon serveur d'autorisation.

- **jwk-set-uri** indique le endpoint qui fournit la clé publique permettant la vérification
- **issuer-uri** pointe vers l'URI du serveur d'autorisation de base. Elle peut être utilisée pour découvrir le endpoint fournissant la clé publique



Exemple application.yml

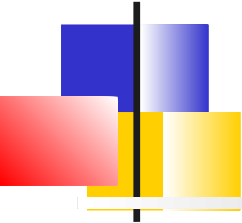
```
server:
  port: 8081
  servlet:
    context-path: /resource-server
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8083/realms/myRealm
          jwk-set-uri: http://keycloak:8083/realms/myRealm/protocol/openid-
connect/certs
```



ACLs typique

@Configuration

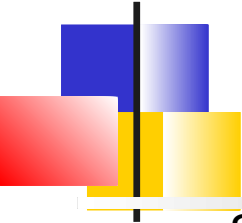
```
public class SecurityConfig {  
    @Bean  
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {  
        http  
            .cors(cors -> cors.disable())  
            .authorizeHttpRequests(auth -> auth  
                .requestMatchers(HttpMethod.GET, "/user/info", "/api/foos/**")  
                    .hasAuthority("SCOPE_read")  
                .requestMatchers(HttpMethod.POST, "/api/foos")  
                    .hasAuthority("SCOPE_write")  
                .anyRequest()  
                    .authenticated()  
            )  
            .oauth2ResourceServer(oauth2 -> oauth2  
                .jwt(jwt -> {})  
            );  
        return http.build();  
    }  
}
```



Spring et les tests

Spring Test

Apports de Spring Boot
Tests auto-configurés



Versions

Spring/SpringBoot/JUnit

SpringBoot 1, Spring 4, JUnit4

Dernière version Septembre 2018, fin de vie 2019

SpringBoot 2, Spring 5, JUnit5

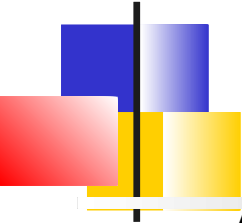
Première version ~2018, fin de vie 2025

SpringBoot 3, Spring 6, JUnit5

Première version Fin 2022

SpringBoot 4, Spring 7, JUnit5

Première version Fin 2025



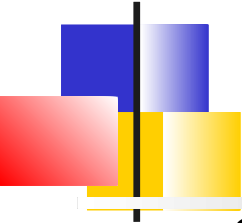
Apport SpringTest

Apporte peu pour le test unitaire

- Mocking de l'environnement en particulier l'API servlet ou Reactive
- Package d'utilitaires : *org.springframework.test.util*

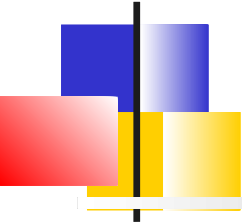
Et beaucoup pour les tests d'intégration (impliquant un ApplicationContext Spring) :

- Cache du conteneur Spring pour accélérer les tests
- Injection des données de test
- Gestion de la transaction (roll-back)
- Des classes utilitaires
- Extension JUnit5, Runner Junit4



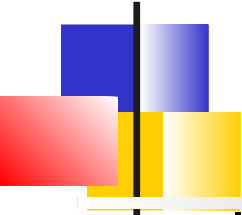
Exemple JUnit5

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = TestConfig.class)
class SimpleTests {
    @Test
    void testMethod() {
        // test logic...
    }
}
```



Spring et les tests

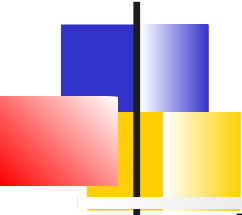
Spring Test
Apports de Spring Boot
Tests auto-configurés



spring-boot-starter-test

L'ajout de ***spring-boot-starter-test*** (dans le scope test), ajoute les dépendances suivantes :

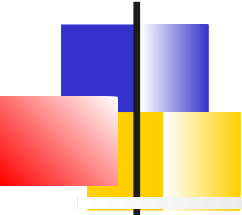
- **Spring Test** : Utilitaires Spring pour le Test
- **Spring Boot Test** : Utilitaire liant Spring Test à Spring Boot
- **Spring Boot Test Autoconfigure** : Tests auto-configurés
- **JUnit5, AssertJ**, (SB 2+):
- **Mockito** : Un framework pour générer des classes Mock
- **JSONAssert** : Une librairie pour les assertions JSON
- **JsonPath** : XPath pour JSON.



Annotations apportées

De nouvelles annotations sont disponibles via le starter :

- **@SpringBootTest** permettant de définir l'ApplicationContext Spring à utiliser pour un test grâce à un mécanisme de détection de configuration
- Annotations permettant des **tests auto-configurés**.
Ex : Auto-configuration pour tester des RestController en isolation
- **@MockitoBean** permettant de créer des beans Mockito

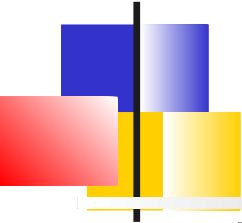


@SpringBootTest

Il est possible d'utiliser l'annotation **@SpringBootTest** pour déterminer le contexte applicatif avant l'exécution du test

L'annotation recherche une classe annotée @SpringBootApplication en remontant dans les packages.

=> Si l'on respecte la hiérarchie des packages, on tombe sur la classe principale de l'application et le contexte applicatif contient tous les beans

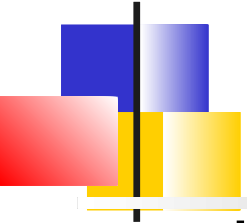


Attribut *classes*

L'annotation `@SpringBootTest` peut également préciser les classes de configuration utilisées pour charger le contexte applicatif via l'attribut `classes`

Exemple :

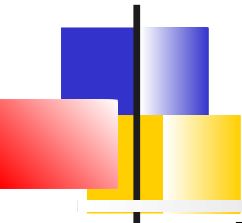
```
@SpringBootTest(classes = ForumApp.class)
```



WebEnvironment

L'attribut ***WebEnvironment*** permet de préciser si l'on désire disposer d'un serveur web lors des tests:

- **MOCK** : Fournit un environnement serveur Mocké
- **RANDOM_PORT** : Le serveur démarré sur un port aléatoire
- **DEFINED_PORT** : Le serveur est démarré sur un port spécifié
- **NONE** : Pas d'environnement servlet.



@MockitoBean

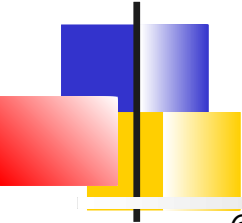
L'annotation **@MockitoBean** définit un bean Mockito

Cela permet de remplacer ou de créer de nouveaux beans

L'annotation peut être utilisée :

- Sur les classes de test
- Sur les champs de la classe de test, dans ce cas le bean mockito est injecté

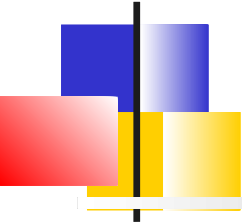
Les beans Mockito sont automatiquement réinitialisés après chaque test



Exemple

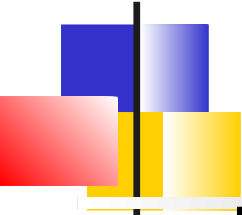
```
@SpringBootTest
public class MyTests {
    @MockitoBean
    private RemoteService remoteService;
    @Autowired
    private Reverser reverser;

    @Test
    public void exampleTest() {
        // RemoteService has been injected into the reverser bean
        given(this.remoteService.someCall()).willReturn("mock");
        String reverse = reverser.reverseSomeCall();
        assertThat(reverse).isEqualTo("kcom");
    }
}
```



Spring et les tests

Spring Test
Apports de Spring Boot
Tests auto-configurés

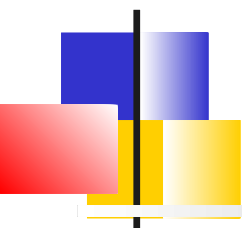


spring-boot-test- autoconfigure

Les capacités d'auto-configuration de Spring Boot peuvent ne pas être adaptées au test.

- Lorsque l'on teste la couche contrôleur, on n'a pas envie que SpringBoot nous démarre automatiquement une base de données

Le module **spring-boot-test-autoconfigure** inclut des annotations qui permettent de tester par couche les applications

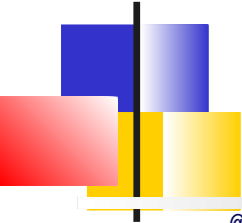


Tester la (dé)sérialisation

Afin de tester si la sérialisation JSON fonctionne correctement, l'annotation **@JsonTest** peut être utilisée.

Elle configure automatiquement l'environnement Jackson ou Gson

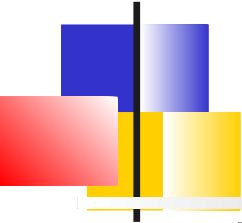
Les classes utilitaires JacksonTester, GsonTester ou BasicJsonTester peuvent être injectées et utilisées, les assertions spécifiques à JSON peuvent être utilisées



Exemple

@JsonTest

```
public class MyJsonTests {
    @Autowired
    private JacksonTester<VehicleDetails> json;
    @Test
    public void testSerialize() throws Exception {
        VehicleDetails details = new VehicleDetails("Honda", "Civic");
        assertThat(this.json.write(details)).isEqualToJson("expected.json");
        assertThat(this.json.write(details)).hasJsonPathStringValue("@.make");
        assertThat(this.json.write(details).extractingJsonPathStringValue("@.make")
            .isEqualTo("Honda"));
    }
    @Test
    public void testDeserialize() throws Exception {
        String content = "{\"make\":\"Ford\",\"model\":\"Focus\"}";
        assertThat(this.json.parse(content))
            .isEqualTo(new VehicleDetails("Ford", "Focus"));
        assertThat(this.json.parseObject(content).getMake()).isEqualTo("Ford");
    }
}
```

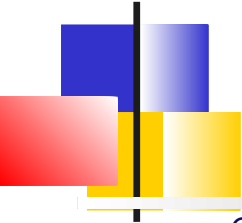


Tests de Spring MVC

L'annotation **@WebMvcTest** configure l'infrastructure Spring MVC et limite le scan aux annotations de Spring MVC

Elle configure également MockMvc qui permet de se passer d'un serveur Http complet

Pour les tests Selenium ou HtmlUnit, un client Web est également fourni



Example

@WebMvcTest(UserVehicleController.class)

```
public class MyControllerTests {
```

```
    @Autowired
```

```
    private MockMvc mvc;
```

```
    @MockitoBean
```

```
    private UserVehicleService userVehicleService;
```

```
    @Test
```

```
    public void testExample() throws Exception {
```

```
        given(this.userVehicleService.getVehicleDetails("sboot"))
```

```
            .willReturn(new VehicleDetails("Honda", "Civic"));
```

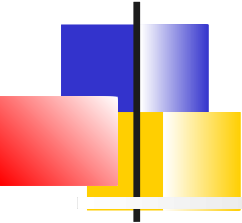
```
        this.mvc.perform(get("/sboot/vehicle").accept(MediaType.TEXT_PLAIN))
```

```
            .andExpect(status().isOk())
```

```
            .andExpect(content().string("Honda Civic"));
```

```
    }
```

```
}
```

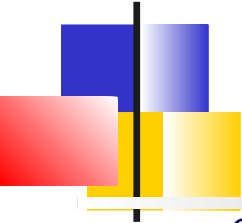
Tests JPA

@DataJpaTest configure une base de donnée mémoire, scanne les *@Entity* et configure les Repository JPA

Les tests sont transactionnels et un rollback est effectué à la fin du test

- Possibilité de changer ce comportement par *@Transactional*

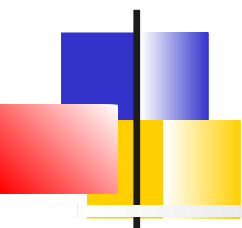
Un *TestEntityManager* peut être injecté ainsi qu'un *JdbcTemplate*



Example

@DataJpaTest

```
public class ExampleRepositoryTests {  
    @Autowired  
    private TestEntityManager entityManager;  
    @Autowired  
    private UserRepository repository;  
    @Test  
    public void testExample() throws Exception {  
        this.entityManager.persist(new User("sboot", "1234"));  
        User user = this.repository.findByUsername("sboot");  
        assertThat(user.getUsername()).isEqualTo("sboot");  
        assertThat(user.getVin()).isEqualTo("1234");  
    }  
}
```



Autres tests auto-configurés

@WebFluxTest : Test des contrôleurs Spring Webflux

@JdbcTest : Seulement la datasource et jdbcTemplate.

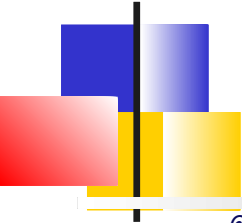
@JooqTest : Configure un DSLContext.

@DataMongoTest : Configure une base mémoire Mongo, MongoTemplate, scanne les classes @Document et configure les MongoDB repositories.

@DataRedisTest : Test des applications Redis applications.

@DataLdapTest : Serveur embarqué LDAP (if available), LdapTemplate, Classes @Entry et LDAP repositories

@RestClientTest : Test des clients REST. Jackson, GSON, ... + RestTemplateBuilder, et du support pour MockRestServiceServer.

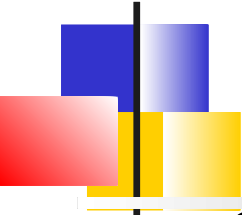


Example

@RestClientTest(RestService.class)

```
public class RestserviceTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
    private ObjectMapper objectMapper;
    @Autowired
    private RestService restService;

    @BeforeEach
    public void setUp() throws Exception {
        Member aMember = ...
        String memberString = objectMapper.writeValueAsString(aMember);
        this.server.expect(requestTo("/members/1"))
            .andRespond(withSuccess(memberString, MediaType.APPLICATION_JSON));
    }
    @Test
    public void whenCallingGetMember_thenOk() throws Exception {
        assertThat(restService.getMember(1)).extracting("email").isEqualTo("d@gmail.com");
    }
}
```



Test et sécurité

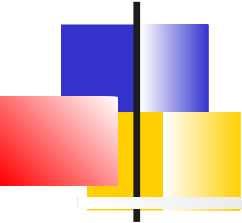
Spring propose plusieurs annotations pour exécuter les tests d'une application sécurisée par SpringSecurity.
Dépendance : ***spring-security-test***

@WithMockUser : Le test est exécuté avec un utilisateur dont on peut préciser les détails (login, password, rôles)

@WithAnonymousUser : Annote une méthode

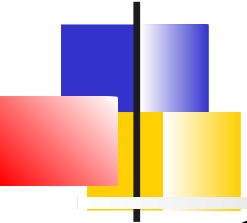
@WithUserDetails("aLogin") : Le test est exécuté avec l'utilisateur chargé par UserDetailsService

@WithSecurityContext : Qui permet de créer le SecurityContext que l'on veut



Vers la production

Actuator
Déploiement



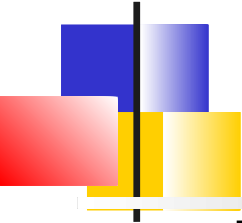
Actuator

Spring Boot Actuator fournit un support pour la surveillance et la gestion des applications SpringBoot

Il peut s'appuyer

- – Sur des points de terminaison HTTP (Si on a utilisé Spring MVC)
- – Sur JMX

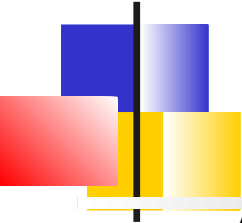
L'activation de Actuator nécessite spring-boot-starter-actuator



Fonctionnalités

Les fonctionnalités transverses offertes par Actuator concernent :

- Statut de santé de l'application
- Obtention de métriques
- Audit de sécurité
- Traces des requêtes HTTP
- Visualisation de la configuration
- ...

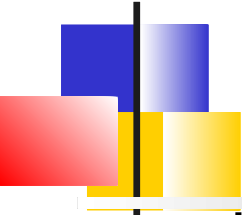


Endpoints

Actuator fournit de nombreux endpoints :

- **beans** : Une liste des beans Spring
- **env / configprops** : Liste des propriétés configurables
- **health** : Etat de santé de l'appli
- **info** : Informations arbitraires. En général, Commit, version
- **metrics** : Mesures
- **mappings** : Liste des mappings configurés
- **trace** : Trace des dernières requête HTTP
- **docs** : Documentation, exemple de requêtes et réponses
- **logfile** : Contenu du fichier de traces

Si on développe un Bean de type Endpoint, il est automatiquement exposé via JMX ou HTTP



Configuration

Les endpoints peuvent être configurés par des propriétés.

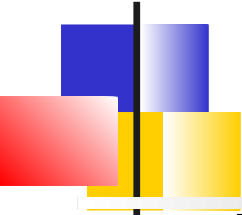
Chaque endpoint peut être

- – Activé/désactivé
- – Sécurisé par Spring Security
- – Mappé sur une autre URL

Seuls les endpoints /health et /info sont activés par défaut

Pour activer les autres :

- – `management.endpoints.web.exposure.include=*`
- – Ou les lister un par un



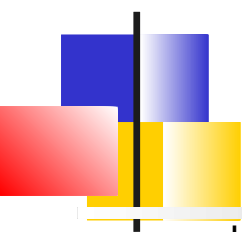
Endpoint /health

L'information fournie permet de déterminer le statut d'une application en production.

- Elle peut être utilisée par des outils de surveillance responsable d'alerter lorsque le système tombe (Kubernetes par exemple)

Par défaut, le endpoint affiche un statut global mais on peut configurer Spring pour que chaque sous-système affiche son statut :

`management.endpoint.health.show-details= always`



Informations sur l'application

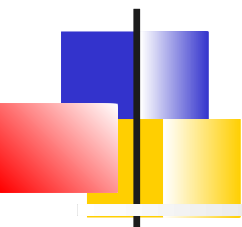
Le endpoint **`/info`** par défaut n'affiche rien.

Si l'on veut les détails sur Git :

`git-commit-id-plugin`

Si l'on veut les informations de build :

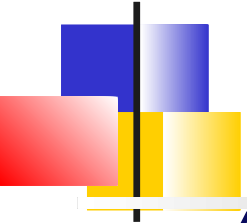
```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution>
      <goals>
        <goal>build-info</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```



Metriques

Le endpoint ***metrics*** donne accès à toute sorte de métriques. On retrouve :

- – Système : Mémoire, Heap, Threads, GC
- – Source de données : Connexions actives, état du pool
- – Cache : Taille, Hit et Miss Ratios
- – Tomcat Sessions



Autres endpoints

/auditevents : Liste les événements de sécurité (login/logout)

/conditions : Remplace /autoconfig, rapport sur l'auto-configuration

/configprops – Les beans annotés par @ConfigurationProperties

/flyway ; Information sur les migrations de BD Flyway

/liquibase : Migration Liquibase

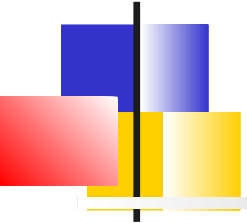
/logfile : Logs applicatifs

/loggers : Permet de visualiser et modifier le niveau de log

/scheduledtasks : Tâches programmées

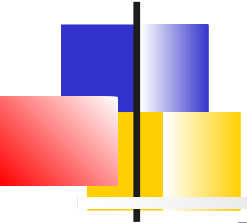
/sessions : HTTP sessions

/threaddump : Thread dumps



Vers la production

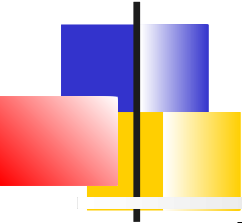
Actuator
Déploiement



Introduction

Plusieurs alternatives pour déployer une application Spring-boot :

- – Application stand-alone
- – Archive war à déployer sur serveur applicatif
- – Service Linux ou Windows
- – Image Docker
- – Le cloud



Application stand-alone

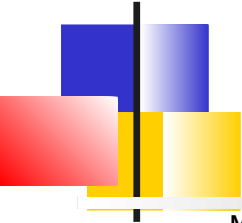
Le plugin Maven de Spring-boot permet de générer l'application stand-alone :

```
mvn package
```

Crée une archive exécutable contenant les classes applicatives et les dépendances dans le répertoire target

Pour l'exécuter :

```
java -jar target/artifactId-version.jar
```



Fichier Manifest

Manifest-Version: 1.0

Implementation-Title: documentService

Implementation-Version: 0.0.1-SNAPSHOT

Archiver-Version: Plexus Archiver

Built-By: dthibau

Start-Class: org.formation.microservice.documentService.DocumentsServer

Implementation-Vendor-Id: org.formation.microservice

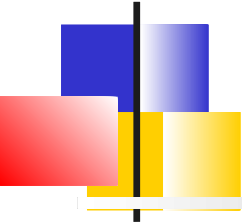
Spring-Boot-Version: 1.3.5.RELEASE

Created-By: Apache Maven 3.3.9

Build-Jdk: 1.8.0_121

Implementation-Vendor: Pivotal Software, Inc.

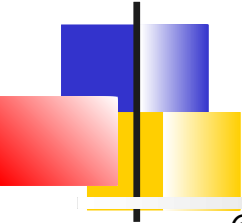
Main-Class: org.springframework.boot.loader.JarLauncher



Création de war

our créer un war, il est nécessaire de :

- Fournir une sous-classe de `SpringBootServletInitializer` et surcharger la méthode `configure()`. Cela permet de configurer l'application (Spring Beans) lorsque le war est installé par le servlet container.
- De changer l'élément `packaging` du `pom.xml` en `war`
- `<packaging>war</packaging>`
- Puis exclure les librairies de tomcat



Exemple

```
@SpringBootApplication
```

```
public class Application extends SpringBootServletInitializer {
```

```
@Override
```

```
protected SpringApplicationBuilder configure(
```

```
SpringApplicationBuilder application) {
```

```
return application.sources(Application.class);
```

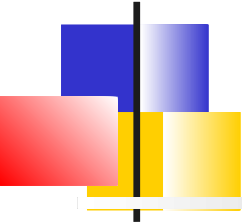
```
}
```

```
public static void main(String[] args) throws Exception {
```

```
SpringApplication.run(Application.class, args);
```

```
}
```

```
}
```

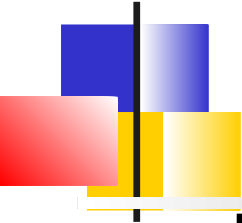


Création de service Linux

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <executable>true</executable>
      </configuration>
    </plugin>
  </plugins>
</build>
```

=> target/artifactId.jar is executable !

```
=> ln -s target/artifactId.jar /etc/init.d/artifact
service artifact start
```

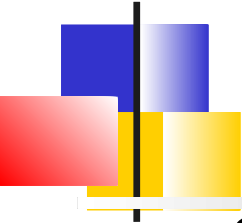


Cloud

Les jars exécutable de Spring Boot sont prêts à être déployés sur la plupart des plate-formes PaaS

La documentation de référence offre du support pour :

- Cloud Foundry
- Heroku
- OpenShift
- Amazon Web Services
- Google App Engine



Exemple CloudFoundry/Heroku

Cloud Foundry

```
cf login
```

```
cf push acloudyspringtime -p target/demo-0.0.1-  
    SNAPSHOT.jar
```

Heroku

Mise à jour d'un fichier Procfile :

```
web: java -Dserver.port=$PORT -jar target/demo-  
    0.0.1-SNAPSHOT.jar
```

```
git push heroku master
```

